March 30th, 2026

Hospital Fixed-Asset Project Report Document

Imperial Healthtech

Supervisors: *Choirunnisa Hapsari*
Kemal Falatehan

Author: Farros Ramzy

Abstract

Hospitals and clinics depend on movable and fixed assets such as wheelchairs, infusion pumps, portable monitors, beds, and reusable support equipment. When these assets are moved without a dependable record, staff can lose time searching for them, the last known location becomes uncertain, and equipment may remain unavailable even though it is still inside the facility. This project developed a working prototype that combines passive UHF RFID tags, ESP32-based Registration Desk and Checkpoint Nodes, a FastAPI backend, PostgreSQL persistence, MQTT messaging, WebSocket live updates, and two role-oriented React applications.

The implemented product uses checkpoint-based location updates rather than continuous coordinate tracking. Assets are registered through the Registration App, associated with RFID tags and their assigned rooms, then detected by fixed checkpoint nodes. The backend validates each event, updates the latest known room, evaluates the asset flow state, records activity and alerts, and broadcasts the result to connected browser clients. A controlled movement workflow separates the request, approval, physical movement, and destination-detection stages so that movement is not considered complete merely because it was approved.

The project demonstrates the feasibility of integrating embedded firmware, asynchronous messaging, centralized business rules, persistent data, and responsive browser interfaces into one maintainable prototype. Evaluation in this report is aligned with the project baseline of 20 user requirements (UR-001 to UR-020), 69 functional requirements (FR-001 to FR-069), 36 non-functional requirements (NFR-001 to NFR-036), 41 use cases (UC-001 to UC-041), and 15 system acceptance criteria (AC-001 to AC-015). The strongest results are the separation of responsibilities between nodes, backend, and frontends; the live synchronization of operational data; the role-based workflow; and the ability to detect wrong-location movement. The main limitations are that the prototype does not provide continuous indoor positioning, has not completed full hospital-site RFID coverage validation, and still requires broader end-to-end, load, recovery, and formal user-acceptance evidence before production use.

Version Table

Version	Date of Creation / Change	Changes	Reviewers	Approval Status
0.0	30/03/2026	Document Initiation: adding the abstract and the introduction of the design.		
0.0.1	31/03/2026	Adding the system overview and High-Level architecture.		
0.5.0	01/04/2026	Adding subsystem designs and initiating the Data design, which is the class diagram.		

Keywords

No.	Keyword
1.	Hospital asset tracking
2.	UHF RFID
3.	ESP32
4.	MQTT
5.	FastAPI
6.	PostgreSQL
7.	React
8.	WebSocket
9.	Checkpoint tracking
10.	Role-based access control

Abbreviations

No.	Abbreviation	Meaning
1.	API	Application Programming Interface
2.	ESP32	Espressif 32-bit microcontroller board/platform
3.	HIS	Hospital Information System
4.	HTTPS	Hypertext Transfer Protocol Secure
5.	JWT	JSON Web Token
6	JSON	JavaScript Object Notation
7.	MQTT	Message Queueing Telemetry Transport
8.	RFID	Radio-Frequency Identification
9.	RBAC	Role-Based Access Control
10.	REST	Representational State Transfer
11.	UHF	Ultra-High Frequency
12.	UI/UX	User Interface / User Experience
13.	WSS	Secure WebSocket

Table of Contents

List of Figures

List of Tables

1. Introduction

This chapter introduces the operational problem, the assignment, the guiding questions, the objectives of the prototype, the project boundaries, and the relationship between this report and the supporting requirements, design, and test documents. The product is an asset-visibility and operational-support system. It is not created to track patients or staff, and it does not make clinical decisions.

1.1. Project Background

Hospitals use movable equipment such as infusion pumps, wheelchairs, portable monitors, patient beds, and other reusable assets across multiple rooms and departments. When these assets are moved without a reliable record, staff may spend unnecessary time searching for equipment, asset utilization becomes harder to evaluate, and misplaced equipment may remain unavailable when needed, especially in Indonesia.

Cases of stolen hospital assets commonly happen in Indonesia because of this very issue. This is supported by many news sources listed on the reference page at the bottom of this document.

The original project plan identified manual tracking methods such as verbal communication, whiteboards, barcode checks, paper logs, and spreadsheets as common sources of outdated information. The project therefore explored an embedded and web-based approach that could connect RFID identification, fixed checkpoints, centralized data processing, and user-facing monitoring.

1.2. Problem Statement

The main operational problem is the absence of a centralized and automatically updated method for connecting asset identity, expected room, latest detected room, node status, and controlled movement information. Manual records and staff memory do not provide dependable live visibility, particularly when equipment moves between rooms or when a sensing node becomes unavailable.

The technical problem is broader than RFID reading. The product must coordinate embedded hardware, Wi-Fi, MQTT messaging, backend validation, database persistence, user authorization, movement-state rules, alerts, and browser updates without allowing one component to bypass the system's business rules. The resulting design therefore treats the backend as the authoritative processing boundary and uses sensor nodes as event sources rather than decision-making devices.

1.3. Project Assignment, Objectives, and Purpose

The project assignment was to design and implement a hospital fixed-asset tracking prototype that combines embedded hardware, wireless communication, backend services, persistent storage, and browser-based monitoring. During development, the initial tracker-and-gateway concept evolved into a checkpoint-based architecture with a dedicated Registration App, a separate Monitor App, and two configurable node roles.

The purpose of the current product is to provide authorized hospital staff with a dependable last-known-location view and a controlled workflow for moving registered assets. The implemented objectives are:

- Register and deregister RFID-tagged assets through a Registration Desk workflow.
- Discover, assign, edit, identify, monitor, unassign, disable or enable where supported, and remove eligible ESP32 node records.

- Use checkpoint detections to update an asset's latest known room and operational flow state.
- Allow monitoring-side users to request or cancel movement and registration-side users to approve or reject requests.
- Complete an approved movement only after detection at the approved destination checkpoint.
- Generate clear warnings for unknown tags, offline nodes, and unauthorized or wrong-location movement.
- Protect actions through authentication, backend RBAC, password security, session handling, and administrator safety rules.
- Synchronize the Registration App and Monitor App through backend WebSocket events after persistent data has been updated.
- Keep firmware, backend, and frontend source code modular enough to be maintained and tested independently.

1.4. Research Questions

The original project plan used the broad guiding question:

"What is the most reliable RFID-based asset tracker system for hospital management systems?"

In this project, the question guided engineering decisions and prototype evaluation; it did not represent a complete academic comparison of every available tracking technology. The related sub-questions focused on communication reliability, indoor operation, continuous system behavior, and integration with a hospital management environment:

Table 1. 4. 1 Research questions and how the project addresses them.

ID	Research questions	How the project addresses it
RQ1	Which communication approach is suitable for reliable indoor node-to-backend communication?	Wi-Fi provides network access, and MQTT provides lightweight asynchronous event transport. Reconnection and resubscription behavior are included in the firmware design.
RQ2	What practical factors affect detection reliability?	Tag orientation, antenna placement, reader range, interference, power quality, node placement, and site layout are treated as installation constraints. Full site validation remains future work.
RQ3	How can asset and node state remain consistent while events occur asynchronously?	The backend validates events, controls state transitions, persists changes, and only then broadcasts live updates.
RQ4	How can the embedded, backend, database, and UI components be integrated maintainably?	Responsibilities are separated into firmware modules, backend routes/services/models, and frontend pages/hooks/services/utilities.

The current implementation answers these questions partially through the chosen architecture and performed unit, simulation, and XSS-attempt tests. Range, power, interference, complete integration, recovery, and performance evidence remain open and are not treated as completed findings in this report.

1.5. Project Scope

The current scope covers the product software, embedded firmware, communication services, database, and browser applications required for checkpoint-based asset tracking. It is intentionally limited to asset visibility and controlled movement rather than continuous indoor positioning.

Table 1. 5. 1 Project scope: in-scope and out-of-scope functions.

No.	In scope	Out of Scope
1.	RFID tag association, asset registration, and deregistration.	Patient or staff tracking.
2.	Registration Desk and Checkpoint Node setup, assignment, heartbeat, status, and supported commands.	Clinical diagnosis, treatment decisions, or medical-equipment control.

3.	Current room, assigned room, checkpoint detections, movement requests, and warnings.	Continuous coordinate-level indoor positioning between checkpoints.

4.	Authentication, roles, user management, password changes, first-admin setup, and system settings.	Full hospital information system, ERP, billing, procurement, or inventory purchasing integration.
5.	MQTT, WebSocket, backend APIs, PostgreSQL persistence, alerts, activity, and configuration.	Guaranteed RFID coverage without site-specific antenna, tag, and interference testing.
6.	Prototype documentation, diagrams, unit tests, simulations, and security-input testing reported so far.	Production certification, full commercial readiness, or unperformed validation activities.

1.6. Project Deliverables

The current deliverables include four implementation codebases, requirements and design documentation, diagram sources, and reported testing activities. The detailed System Test Report is intentionally separate so that test cases and evidence can be maintained without interrupting the project narrative.

Table 1. 6. 1 Project deliverables and current implementation status.

No.	Deliverable	Current Status
1.	FastAPI backend and PostgreSQL data model	Implemented and deployed for prototype use on Render.
2.	Registration App	Implemented as a React/Vite browser application and deployed through HTTPS.
3.	Monitor App	Implemented as a separate React/Vite browser application and deployed through HTTPS.
4.	ESP32 sensor-node firmware	Implemented for Registration Desk and Checkpoint Node behavior using shared configurable modules.
5.	Requirements and Use Case Specification	Maintained as a separate authoritative requirements document.
6.	System Requirements Specification	Maintained separately for system-level requirements and acceptance criteria.
7.	System Design Document	Maintained separately for detailed architecture, class, sequence, state-machine, interface, and deployment design.
8.	Diagram sources	Maintained separately so diagrams can be regenerated and updated.
9.	Testing evidence	Summarized in this report; detailed cases and evidence belong in the separate System Test Report.

1.7. Related Project Documents

Separate planning, requirements, use cases, design, and testing documents support this project. The table identifies the purpose of each document and prevents the duplication of detailed technical content in this report.

Table 1. 7. 1 Supporting project documents and their purposes.

No.	Document	Purpose
1.	Project Plan	Records the original context, assignment, research aspects, phases, timeline, risks, and expected outcomes.
2.	Requirements and Use Case Specification	Defines actors, user requirements, functional and non-functional requirements, use cases, flows, and use-case traceability.
3.	System Requirements Specification	Defines the authoritative system-level requirements, interfaces, data rules, quality attributes, deployment requirements, and acceptance criteria.
4.	System Design Document	Defines architecture, subsystem design, class and component structures, sequence diagrams, state machines, communication design, error handling, security, and deployment design.

1.8. Document Structure

The report first explains the project context, stakeholders, and planning. It then summarizes requirements and technical decisions, describes the architecture and implementation, records the tests completed so far, evaluates the current results, discusses limitations and lessons learned, and concludes with safety, maintenance, residual risk, and future-work considerations.

2. Project Context and Stakeholders

This chapter distinguishes organizational stakeholders from users and technical actors who interact directly with the product under development. A stakeholder may be affected by the project without having a system account, while a system user has a defined role and permission set.

2.1. Hospital Operational Context

Hospital assets move between departments to support changing clinical and operational needs. The prototype is intended to provide a centralized record of registered equipment, its assigned room, its latest checkpoint detection, movement state, and the availability of the nodes that produce tracking information. Registration-side activities are performed through the Registration App and an assigned Registration Desk Node. Monitoring-side activities are performed through the Monitor App. Checkpoint Nodes are installed at known locations and publish detections and heartbeat messages.

The system records a last known checkpoint rather than a continuous route. This distinction is important:

1. An asset may physically be between checkpoints while the system still shows its most recent accepted detection.
2. The user interface therefore combines location data with flow information such as in place, pending approval, on the move, or unauthorized, instead of presenting the last room as an unquestionable real-time coordinate.

2.2. Intended Users

The requirements baseline defines six human roles or setup actors. Their access is intentionally split between administrative, technical, registration, monitoring, and controlled demonstration responsibilities.

Table 2. 2. 1 Intended user classes, primary needs, and typical access.

No.	User class	Primary needs	Typical access
1.	Initial System Administrator	Initialize the product by creating the first Admin account when no administrator exists.	First-Admin and App Setup flow only.
2.	Administrator	Manage users, nodes, assets, movement workflows, dashboards, system settings, and account security.	All authorized product functions in the Registration App and Monitor App.
3.	Test User	Exercise broad operational functions for verification or demonstration without full user-account administration.	Node Management, Asset Management, Monitor App, Change Password, and App Communication Settings.
4.	Technician	Install, discover, assign, identify, edit, unassign, and remove eligible sensor nodes.	Registration App Node Management and Change Password.
5.	Registration Staff	Register and deregister assets and approve or reject pending movement requests.	Registration App Asset Management and Change Password.
6.	Monitor Staff	Observe assets, nodes, alerts, and activity and request or cancel eligible movements.	Monitor App and Change Password page in the Registration App.

2.3. Stakeholders

The project affects more than the users who operate the applications. The following stakeholders influence the requirements, deployment conditions, acceptance decision, maintenance effort, and possible future adoption of the product.

Table 2. 3. 1 Project stakeholders, interests, and influence on acceptance.

No.	Stakeholder	Interest in the project	Influence on acceptance
1.	Hospital or clinic management	Operational visibility, asset availability, cost control, and risk reduction.	Defines whether the product is worth piloting and scaling.
2.	Medical and support staff	Faster discovery of equipment and clear movement status.	Usability and workflow fit determine daily adoption.

3.	Hospital IT department	Network compatibility, security, maintainability, node configuration, and support effort.	Determines technical readiness and deployment feasibility.
4.	Asset or facility management	Accurate inventory identity, room assignment, and movement history.	Validates asset-management rules and reporting needs.

5.	Project supervisors and assessors	Evidence that the project meets technical, process, and documentation expectations.	Review design quality, implementation, testing, and reflection.
6.	Future maintainers	Understandable code, configuration, tests, diagrams, and deployment instructions.	Determine whether the prototype can be safely continued.

2.4. User Roles

Roles control access at two levels. The frontend hides or blocks unavailable pages and actions, while the backend independently rejects restricted API requests. This follows UR-018, FR-009, FR-062, FR-063, NFR-001, and NFR-002: frontend checks improve usability, but backend authorization remains the security boundary.

Table 2. 4. 1 User roles, principal permissions, and important restrictions.

No.	Role	Main permissions	Important restrictions
1.	Initial System Administrator	Create the first Admin account when no administrator exists.	The setup flow becomes unavailable after successful initialization.
2.	Administrator	Manage users, nodes, assets, movement decisions, dashboards, settings, and both applications.	Cannot disable or demote the last active administrator; other self-protection rules apply.
3.	Test User	Use node and asset management, Monitor App functions, password change, and app communication settings for controlled testing.	Does not receive full user-account administration authority and is not a production administrator substitute.
4.	Technician	View and manage eligible nodes, including assignment, editing, blink identification, unassignment, and deletion.	No unrestricted user management, asset registration, movement-decision, or system-setting authority in the requirements baseline.
5.	Registration Staff	View/register/deregister assets and approve or reject movement requests.	No user administration or technical node-management authority.
6.	Monitor Staff	View monitoring pages and request or cancel eligible movement.	Cannot approve or reject movement requests and cannot manage users or nodes.

2.5. Operational Environment

ESP32 nodes operate over Wi-Fi and MQTT. The backend and PostgreSQL run locally or in a cloud demonstration environment; the React frontends use HTTPS and secure WebSocket connections. Real hospital deployment adds RFID and network constraints such as metal, doors, people, antenna placement, tag orientation, reader power, and weak Wi-Fi. Hence, a site survey and controlled pilot remain necessary.

2.6. Assumptions and Constraints

The prototype depends on several network, RFID, hardware, hosting, security, and time-related conditions. The table summarizes each condition and the project response, while distinguishing implemented controls from validation that still has to be completed in a real deployment environment.

Table 2. 6. 1 Project assumptions, constraints, and corresponding responses.

No.	Type	Assumption or constraint	Project response
1.	Network	Nodes and browser clients have usable network coverage.	Reconnect logic exists; deployment still needs coverage validation.
2.	RFID	Each active asset has a compatible readable UHF tag.	Tag association uses the Registration Desk workflow.
3.	Location	A checkpoint represents a room, not a continuous coordinate.	The UI combines last known room with flow status.
4.	Hardware	ESP32, reader, antenna, power, and enclosure impose limits.	Modular non-blocking firmware; installation-specific validation.
5.	Hosting	Low-cost services may sleep, expire, or restrict resources.	External configuration plus backup and migration procedures.
6.	Security	Browser clients and sensor messages are not trusted automatically.	Backend authentication, RBAC, secret handling, and message validation.
7.	Time	Heartbeat and movement logic require consistent timestamps.	Consistent backend storage and defined local display conversion.

3. Requirement and Analysis

The project now has two complementary requirements references. The Requirements and Use Case Specification is authoritative for actors, user requirements, functional and non-functional requirement identifiers, use-case descriptions, and requirements-to-use-case matrices. The System Requirement Document v5.0 is the authoritative system-level baseline for subsystem requirements, interfaces, data rules, safety, deployment, verification methods, and acceptance criteria. This chapter summarizes those documents without copying their complete requirement tables.

3.1. Requirements-Gathering Approach

Requirements gathering followed an iterative engineering process. Initial needs were derived from the hospital asset-location problem and translated into user goals and use cases. During implementation, concrete workflow and technical issues refined the baseline, including the distinction between assigned room and current room, pending placement after registration, movement request and decision states, destination-based completion, offline-node uncertainty, session restoration, responsive layouts, system settings, and the separation of Registration App and Monitor App responsibilities.

The main inputs were the project plan, supervisor and project-review discussions, workflow analysis, implementation constraints, defects found during testing, the Requirements and Use Case Specification, and the System Requirement Document. The final documents formalize those findings as 20 user requirements, 69 functional requirements, 36 non-functional requirements, 41 use cases, and 15 acceptance criteria. Changes to implementation should be reflected in all affected requirements, use cases, design elements, and test evidence rather than updated in only one document.

3.2. User Requirements Summary

The complete user requirement statements remain in the requirements documents. The grouped summary below preserves the authoritative identifiers and shows the main user goals represented by each range.

Table 3. 2. 1 User requirement groups and report-level summaries.

No.	User requirement	Requirement area	Report-level summary
1.	UR-001 to UR-004	Authentication, accounts, and Registration Dashboard	Authorized users can authenticate and change their password; administrators can manage accounts safely; and the Registration Dashboard is role-aware.
2.	UR-005 to UR-007	Node access and management	Authorized technical users can open node management, view node lifecycle states, and assign, edit, unassign, delete, or identify eligible nodes.
3.	UR-008 to UR-010	Asset access and management	Authorized registration-side users can open asset management, register a scanned RFID-tagged asset, and deregister an active asset.
4.	UR-011 to UR-013	Controlled asset movement	Monitor-side users can request or cancel movement; registration-side users can decide requests; approved movement completes only at the approved destination.
5.	UR-014 to UR-015	Operational monitoring and warnings	Monitoring users can view assets, nodes, alerts, activity, and warnings for unknown tags, offline nodes, and unauthorized movement.
6.	UR-016 to UR-017	MQTT input and WebSocket updates	The system receives node scans, detections, and heartbeats through MQTT and broadcasts accepted changes to browser clients.
7.	UR-018	Role protection	Unauthorized roles are prevented from opening restricted pages or performing restricted actions.

8.	UR-019	System settings	Authorized users can view and manage supported MQTT or system configuration settings.
9.	UR-020	First-admin initialization	The first Admin account can be created only when no Admin account exists.

3.3. Functional Requirements Summary

The functional baseline contains FR-001 to FR-069. The ranges below are used throughout the report to avoid inventing a second numbering system.

Table 3. 3. 1 Functional requirement groups and required system behavior.

No.	Functional requirement ID	Feature group	Required behavior
1.	FR-001 to FR-009	Authentication, session, password, users, and role access	Login and account verification, token/role results, logout, own password change, admin user management, last-admin safety, and protected access.
2.	FR-010 to FR-012	Registration Dashboard	Display role-dependent navigation and summaries and hide node or asset information when the role is not permitted.
3.	FR-013 to FR-021	Node management	Open/list nodes, display details, assign/edit, blink through MQTT, process acknowledgment, unassign, and delete eligible records.
4.	FR-022 to FR-030	Asset management	Open/list assets, accept and display registration scans, register unique tags, mark pending placement, and deregister active assets.
5.	FR-031 to FR-041	Asset movement	Request, prevent duplicates, cancel eligible requests, approve/reject, set transit state, complete only at approved destination, and update room/flow data.
6.	FR-042 to FR-051	Monitoring, alerts, and activity	Display dashboards and monitoring data and handle unknown tags, offline nodes, and unauthorized movement warnings.
7.	FR-052 to FR-061	MQTT and WebSocket communication	Receive registration scans, detections, and heartbeats, validate node messages, maintain WebSocket connections, broadcast updates, reconnect, and ignore unsupported events safely.
8.	FR-062 to FR-063	Cross-application role enforcement	Reject unauthorized API actions and hide or block unavailable frontend navigation.
9.	FR-064 to FR-068	System settings	Open settings, retrieve values, allow Admin or authorized Test User updates, validate values, and reject unauthorized changes.
10.	FR-069	First-admin setup	Create the first Admin account only under the no-admin setup condition.

3.4. Non-Functional Requirements Summary

The quality baseline contains NFR-001 to NFR-036. Mandatory statements use “shall,” while recommended or target behavior uses “should.” This distinction is retained when evaluating the prototype.

Table 3. 4. 1 Non-functional requirement groups and quality interpretations.

No.	Non-functional requirement ID	Quality attribute	Report-level interpretation
1.	NFR-001 to NFR-008	Security	Authentication, role restriction, password hashing, disabled-account rejection, administrator safety, session clearing, HTTPS, and protection of sensitive configuration.
2.	NFR-009 to NFR-013	Reliability	Stored data remains available when live updates fail; clients reconnect; nodes time out; MQTT messages are validated; unsupported conditions do not crash the application.
3.	NFR-014 to NFR-015	Availability	The backend should support expected operating hours, and the frontend must show clear service or connection failures.
4.	NFR-016 to NFR-018	Performance	Reasonable page loading, timely event propagation, and usable search, sorting, filtering, and pagination.
5.	NFR-019 to NFR-024	Usability	Clear statuses and feedback, role-appropriate actions, responsive desktop/half-window/tablet/mobile layouts, and workflows understandable without protocol knowledge.
6.	NFR-025 to NFR-028	Maintainability and testability	Small, readable modules, reuse through services/hooks/utilities, consistent terminology, and tests for critical backend workflows.
7.	NFR-029 to NFR-030	Compatibility	Modern browser support and compatibility with the selected ESP32, UHF reader, Wi-Fi, and MQTT environment.
8.	NFR-031 to NFR-034	Data integrity	No duplicate active tag, consistent room/movement/flow state, destination-only movement completion, and consistent status updates.
9.	NFR-035	Auditability	Maintain understandable activity records for significant asset, node, movement, and warning events.
10.	NFR-036	Scalability	Additional nodes and assets should be added without a major architectural redesign.

3.5. Main Use Cases

The Use Case Specification defines 41 use cases. The report groups them by feature area; detailed preconditions, normal flows, alternatives, and postconditions remain in the reference document.

Table 3. 5. 1 Main use-case groups, principal actors, and expected outcomes.

No.	Use-case range	Feature group	Principal actors and outcome
1.	UC-001 to UC-005	Account Authentication and Management	Initial System Administrator and authenticated roles initialize the system, log in/out, change their password, and allow Admin user management.
2.	UC-006 to UC-008	Common Registration App Access	Authorized roles open the role-based dashboard, password page, or Admin-only user page.
3.	UC-009 to UC-010	Node and Asset Registration Access	Admin/Test User/Technician open Node Registration; Admin/Test User/Registration Staff open Asset Registration.
4.	UC-011 to UC-016	Node Management	Admin, Test User, and Technician view, assign, edit, blink, unassign, and delete eligible nodes.
5.	UC-017 to UC-020	Asset Management	Admin, Test User, and Registration Staff view/register/deregister assets; the Registration Desk Node sends the scan.
6.	UC-021 to UC-022	Asset Movement Request	Admin, Test User, and Monitor Staff request or cancel eligible movement.
7.	UC-023 to UC-025	Asset Movement Management	Admin, Test User, and Registration Staff approve/reject; the backend completes approved movement after destination detection.
8.	UC-026 to UC-030	Asset Tracker Monitor	Authorized monitor-side roles view dashboard, assets, nodes, current status, and live updates.
9.	UC-031 to UC-035	Asset Alert and Activity	Monitoring roles view alerts/activity and see unknown-tag, offline-node, and unauthorized-movement warnings.
10.	UC-036 to UC-040	Node Realtime Communication	Registration Desk Node, Checkpoint Node, MQTT broker, and backend deliver scans, detections, heartbeats, MQTT messages, and WebSocket updates.
11.	UC-041	System Settings	Admin and Test User view or update supported system settings according to validation and authorization rules.

3.6. Requirements Prioritization

The attached requirements documents do not assign a separate MoSCoW identifier to each requirement. The authoritative priority signal is the normative wording and its acceptance relationship. “Shall” statements are mandatory for the defined baseline; “should” statements are expected quality or deployment targets whose evidence may remain incomplete at prototype stage; and out-of-scope functions are explicitly excluded rather than treated as failed requirements.

Table 3. 6. 1 Requirements prioritization classifications used in this report.

No.	Classification	Meaning in this report	Examples
1.	Mandatory baseline	A “shall” requirement must be implemented and verified before it can be marked accepted.	Authentication and RBAC, unique active tag, movement-state rules, node-message validation, clear errors, and data consistency.
2.	Recommended/target behavior	A “should” requirement is evaluated and planned, but full production evidence may remain open in the prototype.	HTTPS production deployment, normal loading time, maintainable modules, modern browser support, audit history, and scalability.
3.	Outside the baseline	The function is excluded from the current product boundary and is not counted as an unmet requirement.	Continuous indoor coordinates, patient/staff tracking, medical-device control, predictive analytics, and full HIS/ERP integration.

3.7. Requirements Traceability Approach

Traceability is maintained through the sequence UR → FR/NFR → UC → design element → implementation module → test case or acceptance criterion. The Requirements and Use Case Specification contains the UR/FR/NFR-to-UC matrices. The System Requirement Document adds system acceptance evidence through AC-001 to AC-015. The System Design Document shows realization, and the System Test Report should record execution evidence. The compact table below is a navigation summary, not a replacement for either authoritative matrix.

Table 3. 7. 1 Summary traceability from requirements to use cases and acceptance criteria.

No.	Requirement area	User requirement	Functional/quality requirement	Use case	Acceptance criteria
1.	Initialization, authentication, and accounts	• UR-001 to UR-004 • UR-018 • UR-020	• FR-001 to FR-012 • FR-062 to FR-063 • FR-069 • NFR-001 to NFR-008	• UC-001 to UC-010 • UC-026 • UC-041	• AC-001 • AC-002
2.	Node management and identification	• UR-005 to UR-007	• FR-013 to FR-021	• UC-009 • UC-011 to UC-016	• AC-003 • AC-004
3.	Asset registration lifecycle	• UR-008 to UR-010	• FR-022 to FR-030 • NFR-031	• UC-010 • UC-017 to UC-020	• AC-005
4.	Movement request and completion	• UR-011 to UR-013	• FR-031 to FR-041 • NFR-032 to NFR-034	• UC-021 to UC-025	• AC-006 • AC-007
5.	Monitoring and warnings	• UR-014 to UR-015	• FR-042 to FR-051 • NFR-019 to NFR-024 • NFR-035	• UC-026 to UC-035	• AC-008 • AC-009 • AC-010 • AC-013
6.	MQTT and live updates	• UR-016 to UR-017	• FR-052 to FR-061 • NFR-009 to NFR-013	• UC-030 • UC-036 to UC-040	• AC-011
7.	System settings	• UR-019	• FR-064 to FR-068	• UC-041	• AC-012
8.	Failure handling and deployment	• Cross-cutting	• ERR-001 to ERR-015 • DEP-001 to DEP-015	• Multiple operational use cases	• AC-014 • AC-015

4. Research and Technology Selection

The operational problem, prototype scope, available hardware, maintainability, and integration effort guided technology selection. The project did not select technology solely because it was familiar; each choice had to fit the checkpoint-based architecture and the need to connect sensor events to secure browser workflows.

4.1. RFID Tracking Approaches

The project compared several identification and localization approaches against the need for automatic detection, manageable infrastructure cost, low tag maintenance, and achievable prototype accuracy. The comparison below explains why passive UHF RFID was selected for checkpoint-based tracking.

Table 4. 1. 1 Comparison of candidate asset identification and localization approaches.

No.	Approach	Strengths for this project	Limitations for this project
1.	Barcode / QR	Low tag cost and simple software.	Requires line-of-sight and deliberate scanning; unsuitable for automatic checkpoint detection.
2.	Passive UHF RFID	Contactless, no tag battery, can detect assets within a configured reader area, and supports many tags.	Affected by antenna placement, metal, orientation, power, and site layout; provides a detection zone rather than a coordinate.
3.	Bluetooth Low Energy beacons	Can provide periodic radio presence and approximate proximity.	Battery maintenance and beacon management increase operational burden.
4.	UWB / advanced localization	Can support more precise positioning in a designed environment.	Higher infrastructure cost, calibration complexity, and scope beyond this prototype.

4.2. Checkpoint-Based Tracking Decision

Checkpoint tracking was selected because the project needed a realistic prototype that could identify when an asset was observed at a known location without claiming continuous positioning. Each checkpoint node has a stable identity and an assigned hospital room. When it detects a known tag, the backend associates the observation with that location and updates the asset's latest known state.

This approach reduces infrastructure complexity and keeps the location model understandable to users. Its main trade-off is that it cannot describe the path between checkpoints. The report therefore uses the term "last known location" and combines it with movement state and timestamp information.

4.3. UHF RFID Selection

Passive UHF RFID was selected for the asset tags because tags do not require a battery and can be read without direct visual alignment. The technology fits the registration and doorway/checkpoint concept better than barcode scanning. It also allows the same tag identifier to be used across registration and movement detection workflows.

The selection does not remove the need for site testing. Reader power, antenna polarization, tag mounting, asset material, nearby metal, people, and overlapping read zones can all affect detections. The prototype demonstrates system integration, while production acceptance requires controlled placement and coverage tests.

4.4. ESP32 Hardware Selection

ESP32 was selected as the node controller because it provides Wi-Fi connectivity, sufficient processing capability for the prototype, serial interfaces for the UHF reader, and a mature development ecosystem. PlatformIO supports repeatable builds, dependency management, and separate native or embedded test environments.

The hardware still imposes constraints. Power instability, memory limits, blocking code, and network reconnection can make an otherwise correct design unreliable. The firmware was therefore divided into network, configuration, MQTT, RFID, timing, LED, and board-control responsibilities, with non-blocking runtime behavior wherever practical.

4.5. MQTT Communication Selection

Indonesia's healthcare digital-transformation strategy increasingly emphasizes integrated, interoperable, and connected health services. The Ministry of Health has identified fragmented applications, duplicated data collection, inconsistent metadata, and non-standard integration as important weaknesses in the existing digital-health ecosystem. Its Smart Hospital 5.0 direction also places connected medical and operational devices within the Internet of Medical Things as an important part of future hospital development.

Research conducted in Indonesian hospitals further indicates that IoT can support process improvement, reduce manual monitoring effort, and improve operational visibility. Indonesian researchers have examined IoT implementation in public and private hospitals in Jakarta, while another Indonesian study successfully used MQTT to transmit infusion-monitoring data to medical personnel.

MQTT is suitable for this type of hospital IoT communication because it is a lightweight publish/subscribe protocol designed for constrained devices and limited-bandwidth environments. It separates sensor publishers from backend subscribers, supports different message-delivery quality levels, minimizes protocol overhead, and can report abnormal disconnections. These

characteristics are appropriate for applications such as RFID asset tracking, equipment-status monitoring, environmental sensing, node heartbeat reporting, and operational alerts.

MQTT should therefore be considered as an internal operational messaging technology rather than a replacement for Indonesia's clinical-data interoperability standards. SATUSEHAT uses standardized health-data exchange mechanisms, including FHIR-based APIs, while MQTT can transport raw device and sensor events to a secured backend before validated information is stored or exchanged with other hospital systems. This combined architecture allows hospitals to benefit from responsive IoT communication while preserving centralized validation, authorization, auditability, and data interoperability.

MQTT was selected for communication between sensor nodes and the backend because it supports lightweight publish/subscribe messaging, asynchronous events, targeted commands, and decoupling between node firmware and backend processing. The broker distributes messages, while the backend subscribes to the configured hospital and node topic namespace.

MQTT is not exposed directly to the browser applications. This decision centralizes topic validation, node identity checks, business rules, persistence, and live event generation inside the backend. It also avoids distributing broker credentials to frontend clients.

4.6. FastAPI and PostgreSQL Selection

FastAPI was selected for the backend because it supports typed request validation, dependency-based authentication and authorization, asynchronous interfaces, and clear API organization. SQLAlchemy separates persistent models and database operations from HTTP route handling. PostgreSQL provides relational constraints, transactions, indexing, and a suitable foundation for users, nodes, assets, detections, movements, alerts, activity, and settings.

The principal trade-off is operational complexity: the backend, database, MQTT connection, and WebSocket service must remain coordinated. The project addresses this through modular services, environment-based configuration, transaction handling, and health/status feedback.

FastAPI also supports bearer-token and JWT authentication patterns and persistent WebSocket endpoints. Independent benchmarks indicate strong performance among Python frameworks under tested conditions, although this project's performance also depends on PostgreSQL, SQLAlchemy, MQTT processing, network conditions, and the implemented business rules.

4.7. React and Vite Selection

React was selected for both web applications because the interfaces contain reusable tables, forms, badges, modal workflows, role-aware navigation, and real-time state updates. Vite provides a lightweight development and production build process. The two-app design keeps registration and operational monitoring responsibilities distinct even though they share the same backend.

The frontend structure uses page components for composition, custom hooks for data and interaction logic, service modules for API calls, utilities for formatting and state normalization, and shared components for common UI behavior. This avoids placing all logic in large page files and supports responsive improvements without rewriting the entire application.

4.8. WebSocket Live-Update Selection

WebSocket was selected for server-to-browser live updates after the backend has accepted and persisted an event. It is used for node status, asset state, scan, movement, alert, and activity updates. HTTP APIs remain the source for initial loading and recovery; the socket supplements rather than replaces the persistent source of truth.

This hybrid approach is important because WebSocket messages can be missed during a temporary disconnect. After reconnection, the frontend can reload current data through the API to restore consistency.

4.9. Cloud Hosting Selection

Cloud hosting was used to make the backend, database, and frontend applications accessible for remote demonstration and review. Render was selected for the backend and PostgreSQL prototype deployment, while static frontend deployments use HTTPS environment configuration for the API and WebSocket addresses.

Free or limited hosting services may sleep, expire, or restrict connection resources. Cloud deployment is therefore treated as a prototype convenience rather than proof of production readiness. Backup, migration, monitoring, broker security, domain management, and service-level expectations must be addressed before operational use.

4.10. Technology Trade-Offs

No technology selected for the prototype is optimal in every dimension. The following table records the principal benefit of each design decision together with the limitation that was knowingly accepted, so the selected architecture can be evaluated transparently.

Table 4. 10. 1 Technology decisions, benefits, and accepted trade-offs.

No.	Decision	Benefit	Accepted trade-off
1.	Checkpoint tracking instead of continuous localization	Lower infrastructure complexity and clearer prototype scope.	Location is only updated at known checkpoints.
2.	Two frontend applications	Separates registration and monitoring workflows.	Some components and authentication behavior must be kept consistent in two codebases.
3.	Backend-authoritative MQTT processing	Centralizes business rules and security.	The backend is a critical dependency for all accepted node events.
4.	WebSocket plus HTTP recovery	Responsive live UI with a reliable persistent fallback.	More connection-state logic is required in both applications.
5.	Cloud prototype hosting	Easy remote access for review and demonstration.	Free-tier limitations and internet dependency reduce operational reliability.

5. Project Approach and Management

The project was managed as an iterative embedded-and-web development effort. Because behavior crossed hardware, network, backend, database, and frontend boundaries, implementation and validation were performed in small vertical slices rather than completing one technology layer in isolation.

5.1. Development Method

An iterative prototyping method was used. Each iteration selected a concrete workflow—such as node discovery, registration scanning, asset movement, heartbeat handling, or live browser update—then implemented the necessary firmware, backend, database, and UI changes. Defects discovered during integration were used to refine requirements and structure.

This method was suitable because several important rules became clear only when the complete workflow was exercised. Examples include the distinction between approval and movement completion, the need to prevent duplicate pending requests, the handling of offline registration nodes, and the need to reload data after a WebSocket interruption.

5.2. Project Phases

The work progressed through connected phases rather than isolated technology tasks. Outputs from the problem and requirements phases guided architecture and implementation, while integration defects could trigger revisions to earlier requirements, diagrams, or implementation decisions.

Table 5. 2. 1 Project phases, principal activities, and outputs.

No.	Phase	Main activities	Primary output
1.	Problem and scope	Operational problem, assignment, initial research questions, in/out scope.	Project plan and early concept.
2.	Requirements	Users, roles, use cases, functional and quality requirements.	Requirements and Use Case Specification; SRS.
3.	Architecture	Component boundaries, communication, data model, state behavior, deployment.	System Design Document and diagram sources.
4.	Core implementation	Firmware, backend, database, Registration App, Monitor App.	Working vertical workflows.
5.	Integration and refinement	MQTT/WebSocket integration, movement rules, role access, responsive UI, refactoring.	Integrated prototype and reduced defects.
6.	Verification and reporting	Unit tests, simulations, manual system checks, documentation, evaluation.	System Test Report evidence and final project report.

5.3. Planning and Milestones

Planning used milestones rather than treating every implementation task as equally critical. The central milestones were a functioning node-to-backend event path, a persistent asset model, registration workflow, checkpoint location update, controlled movement workflow, role-protected applications, live updates, deployment, and final verification.

5.4. Task and Version Management

Source code was separated into firmware, backend, Registration App, and Monitor App repositories or project folders. Git was used for version control, with environment-specific secrets excluded from tracking. Refactoring work focused on reducing large files, moving repeated behavior into services or utilities, and keeping changes reviewable without unnecessarily altering visible behavior.

Document versions were maintained separately from source-code versions. Requirements, design, test, and report documents have different purposes; therefore, a change to implementation may require corresponding updates in more than one document rather than copying the same technical text into every deliverable.

5.5. Risk Management

Risks were reviewed across physical sensing, electrical power, network communication, cloud hosting, access control, and cross-application state consistency. The table records the expected impact and the current mitigation, while recognizing that several risks still require site or production validation.

Table 5. 5. 1 Project risks, impacts, and current mitigation measures.

No.	Risk	Impact	Mitigation / current response
1.	RFID blind spots or repeated reads	Incorrect or missing location events.	Tag/read cooldown, site-specific antenna placement, controlled range testing, and explicit last-known-location wording.
2.	Weak or unstable node power	Unexpected resets, reader failure, or intermittent communication.	Use a suitable regulated supply, validate current capacity, and separate power troubleshooting from software faults.
3.	Wi-Fi or MQTT interruption	Nodes stop reporting or commands are not acknowledged.	Reconnect, resubscribe, heartbeat status, offline detection, and visible node state.

4.	Cloud service expiry or sleep	Backend/database becomes temporarily unavailable.	Backups, migration procedure, external configuration, and production hosting review.
5.	Role or route misconfiguration	Unauthorized actions or blocked legitimate users.	Backend RBAC tests, role matrices, frontend route guards, and access review.
6.	State inconsistency across apps	Users see conflicting asset or node status.	Backend source of truth, shared event semantics, WebSocket recovery reload, and normalized frontend status logic.

5.6. Change Management

Changes were evaluated against four questions: Does the change solve a real project or user problem? Does it preserve the backend as the authoritative rule boundary? Can the change be represented consistently across both frontends and firmware where relevant? Can it be tested without invalidating completed workflows? Changes that expanded scope without supporting the core objective were deferred.

Significant changes included moving from a single monitor concept to separate Registration and Monitor applications, introducing request/approval/completion movement states, refining node lifecycle behavior, adding live updates, improving session persistence, introducing multi-hospital topic considerations, and restructuring oversized source files.

5.7. Communication and Review Process

Progress was reviewed through implementation demonstrations, screenshots, source-code inspection, test results, and document updates. Feedback was translated into specific actions such as restoring role access, correcting inconsistent counts, improving responsive layouts, preventing duplicate submissions, and clarifying state names. For final assessment, the report should be reviewed together with the deployed demonstration, code repositories, design diagrams, and test evidence.

6. System Architecture and Design Summary

This chapter summarizes the architecture needed to understand the implementation and evaluation. Detailed class diagrams, module diagrams, sequence diagrams, state machines, and interface definitions remain in the System Design Document so that this report can focus on the project narrative and results.

6.1. System Overview

The product consists of RFID-tagged assets, ESP32 sensor nodes, an MQTT broker, a FastAPI backend, a PostgreSQL database, and two React applications. A Registration Desk Node reports scanned tags used by registration workflows. Checkpoint Nodes report detections and heartbeat status. The backend validates events, applies business rules, stores data, generates alerts and activity records, and broadcasts accepted changes to the browsers.

The system separates observation from decision. Firmware reports what it detected and its current node status. The backend decides whether the message belongs to a configured node, whether the tag matches an active asset, whether a movement is authorized, and which data or alert should change. This prevents firmware or frontend code from bypassing the central workflow.

6.2. High-Level Architecture

The figure below summarizes the original operating concept of the product. RFID-tagged assets are registered at a dedicated station and are later detected by room-based checkpoints. In the implemented architecture, the FastAPI backend mediates MQTT processing, database updates,

authorization, and browser synchronization; the frontend applications do not write directly to the database or communicate directly with the MQTT broker.

The diagram traces the basic asset journey from registration to room detection. A tag is scanned at the registration table, checkpoint readers report later observations through Wi-Fi and MQTT, and the system updates the asset's latest accepted checkpoint. This view is conceptual; the implemented backend and database boundaries are described in the following subsections.

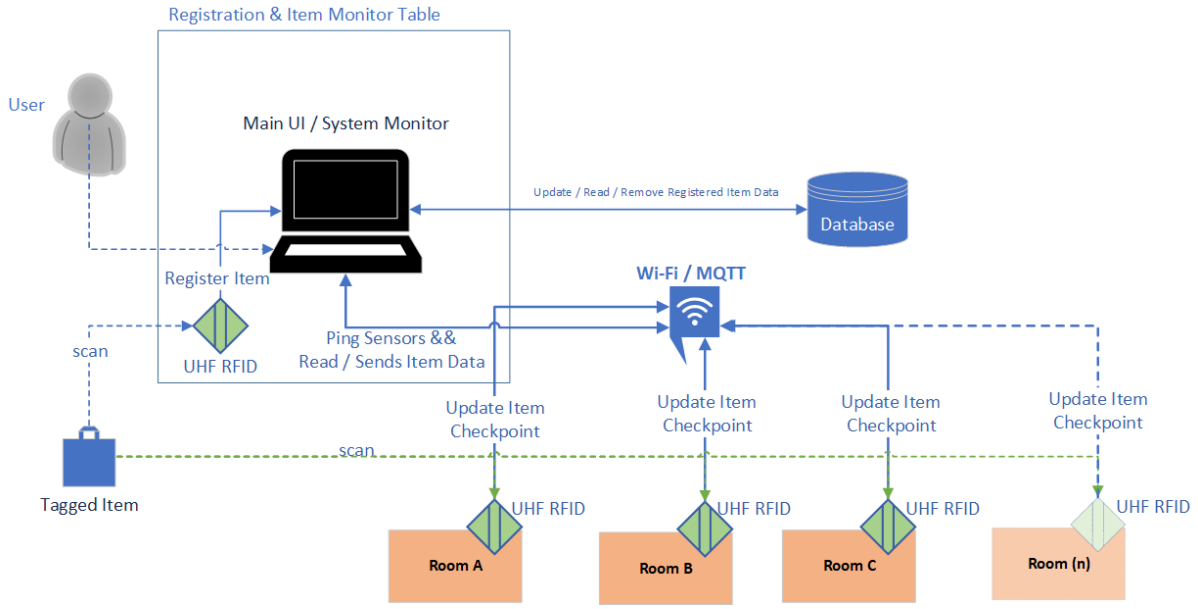

Figure 6. 2. 1 High-level architecture and checkpoint-based asset-tracking flow.

6.3. Main System Components

The architecture assigns a clear responsibility to each physical and software component. The table summarizes ownership and interfaces so that sensing, message transport, business processing, persistence, and user interaction remain separated.

Table 6. 3. 1 Main system components, responsibilities, and interfaces.

No.	Component	Responsibility	Primary interfaces
1.	Registration Desk Node	Read a tag during registration/deregistration and publish a normalized scan event.	UHF reader, Wi-Fi, MQTT.
2.	Checkpoint Node	Detect RFID tags in an assigned room, publish detections and heartbeat, receive supported commands.	UHF reader, Wi-Fi, MQTT.
3.	MQTT broker	Route sensor events and backend commands according to configured topics.	MQTT publish/subscribe.
4.	FastAPI backend	Authenticate users, authorize actions, validate events, apply business rules, persist state, generate events.	HTTPS REST, MQTT client, WebSocket, PostgreSQL.
5.	PostgreSQL database	Store operational and historical records with relational integrity.	SQLAlchemy/PostgreSQL connection.
6.	Registration App	Node management, asset registration/deregistration, movement decisions, users, account security, settings.	HTTPS and secure WebSocket.
7.	Monitor App	Dashboard, assets, nodes, alerts, activity, and movement-request creation.	HTTPS and secure WebSocket.

6.4. Embedded Firmware Design Summary

The ESP32 firmware supports both Registration Desk and Checkpoint roles through shared configurable modules. NodeManager coordinates startup and recurring behavior. Network configuration and captive-portal logic manage Wi-Fi setup. The MQTT module manages broker connection, subscriptions, publishing, commands, and acknowledgments. The RFID module reads and filters tag observations. Clock, LED, and board-control abstractions support timing, status indication, blink identification, and reset behavior.

The firmware is designed to recover from ordinary network interruptions without requiring a manual reset. After MQTT reconnection, required command topics are subscribed again. Repeated observations of the same tag are suppressed for a short interval so that one physical presence does not create excessive duplicate events.

The firmware relationship diagram shows NodeManager as the coordinating module for Wi-Fi, stored and remote configuration, MQTT communication, RFID acquisition, timing, LED status, and board-level control. The use of dedicated interfaces allows the same runtime structure to support different node roles and makes hardware-dependent behavior easier to isolate during testing.

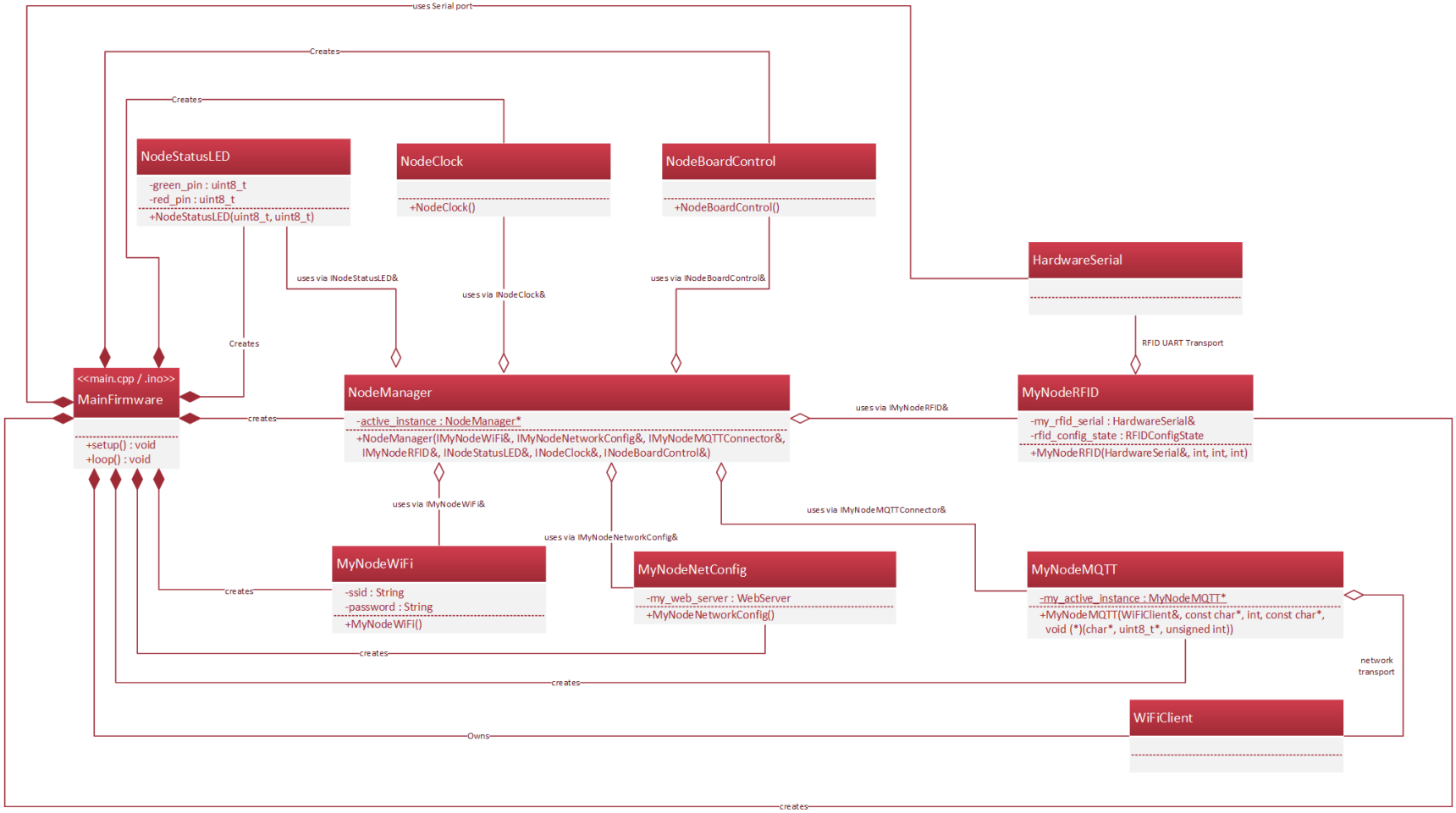

Figure 6. 4. 1 Firmware module relationships and NodeManager coordination.

6.5. Backend Design Summary

The backend is organized into FastAPI route modules, authentication dependencies, request/response schemas, service modules, repositories or data-access functions, SQLAlchemy models, MQTT processing, background status monitoring, and WebSocket broadcasting. Routes remain thin and delegate business decisions to services so that rules can be tested outside the HTTP layer.

The main service areas are authentication and users, node discovery and assignment, asset registration, movement request processing, checkpoint location updates, flow-state resolution, alerts, activity, settings, and live events. Transactions are used so that an operation either commits a consistent set of changes or rolls back when validation or persistence fails.

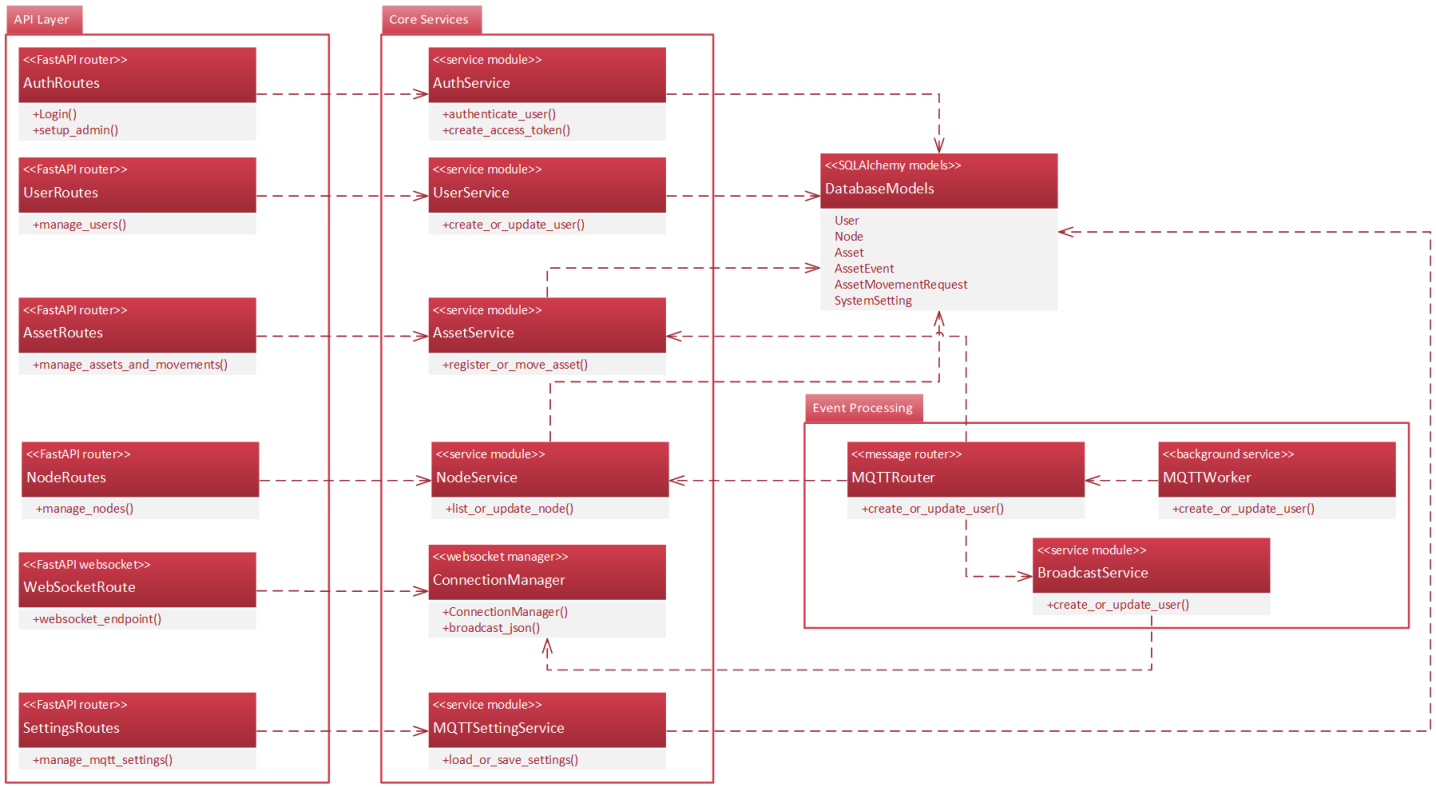

Figure 6. 5. 1 Backend layered architecture and event-processing flow.

The backend diagram above separates HTTP routes, core services, SQLAlchemy models, MQTT event processing, and WebSocket broadcasting. User requests and sensor events enter through different interfaces but are routed into centralized services before data is committed, which keeps authentication, validation, movement rules, alerts, and live updates consistent.

6.6. Registration App Design Summary

The Registration App is the administration and registration-side interface. It contains role-protected routes for the dashboard, node registration, asset registration, movement approval, user management, account security, and system settings where enabled. Page components focus on composition, while hooks own loading and interaction state, services perform API calls, and utilities normalize dates, text, status, sorting, pagination, and session behavior.

The app combines initial API loading with WebSocket events. It provides loading feedback, error banners, responsive tables, role-aware actions, and session restoration. Offline nodes are prevented from performing actions that require active communication, and backend errors remain authoritative even when the frontend has already validated a form.

6.7. Monitor App Design Summary

The Monitor App is optimized for operational visibility. Its dashboard summarizes assets, node availability, alerts, and recent activity. Detailed pages provide searchable, sortable, and paginated tables for assets, nodes, alerts, and activity. The asset page shows assigned room, current room, last seen time, and flow status, and allows authorized users to request or cancel movement.

Shared hooks and utilities keep status labels and filtered counts consistent across overview and detail pages. Connection indicators distinguish API loading and WebSocket connectivity from the actual online/offline state of sensor nodes. After a socket reconnect, the app can reload current backend data to correct missed events.

6.8. Data Model Summary

The logical data model includes users, system settings, hospitals or hospital identifiers, rooms or location values, nodes, assets, detection events, movement requests, alerts, and activity events. The physical implementation may store some hospital or room information as validated fields rather than separate master tables; the design document should state the exact database representation.

Important integrity rules include one active asset per active tag, no more than one pending movement request per asset, stable node identity, conditionally required room values for Checkpoint Nodes, and completion of movement only when a detection comes from the approved destination. Timestamps support last seen, heartbeat timeout, decision history, and activity ordering.

The logical data model groups identity and configuration, hospital location and nodes, asset tracking, movement requests, detections, alerts, and activity records. Its relationships support traceability from a user or node action to the affected asset, persistent state transition, warning, and historical event. Some hospital or room concepts may be implemented as validated fields rather than separate physical tables.

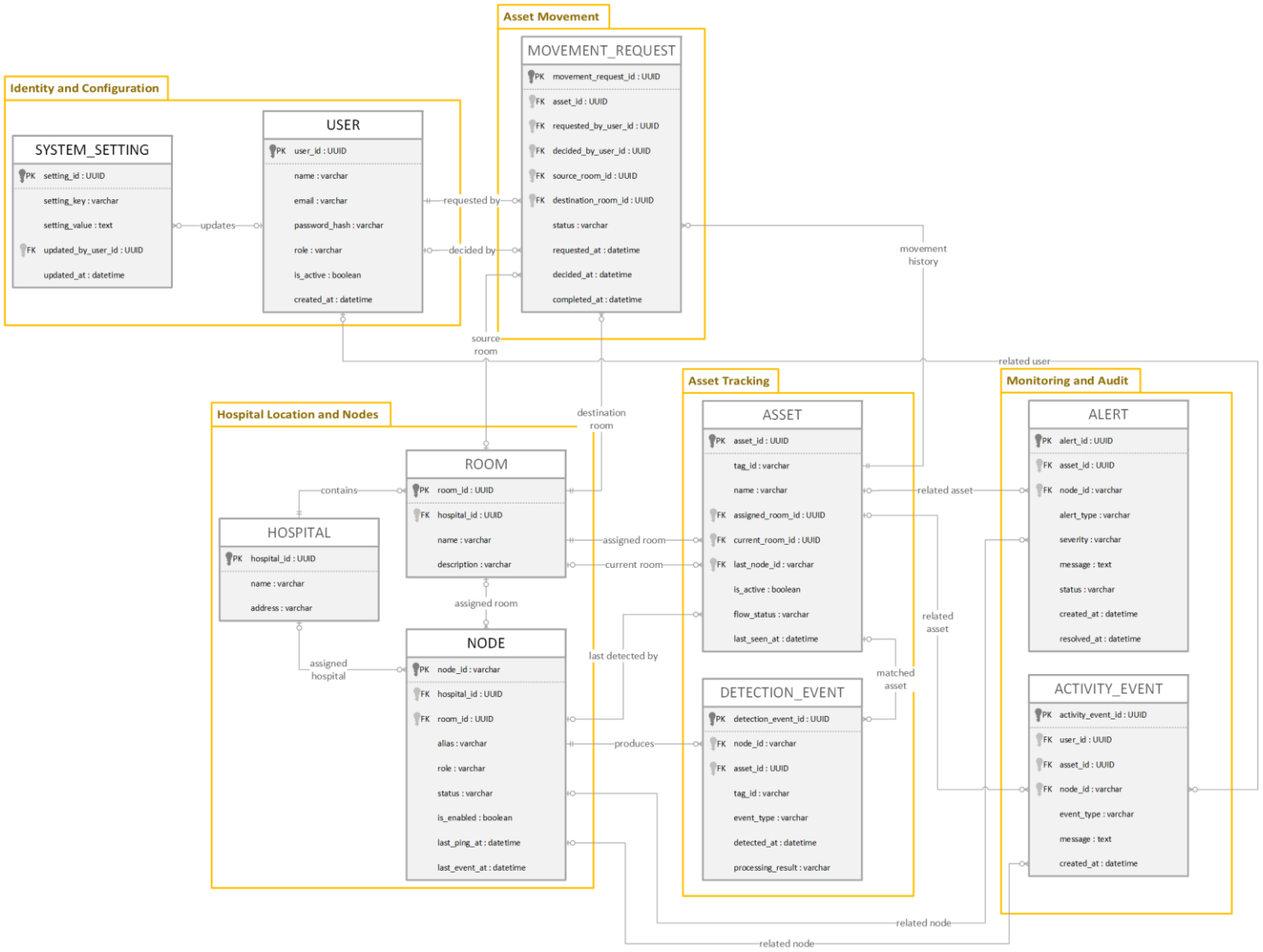

Figure 6. 8. 1 Logical data model and main entity relationships.

6.9. Communication Design Summary

The communication design combines synchronous requests with asynchronous operational events. The figure shows the end-to-end relationship between RFID-tagged assets, sensor nodes, the MQTT broker, the backend, persistent storage, and both browser applications; the table then summarizes the purpose of each protocol.

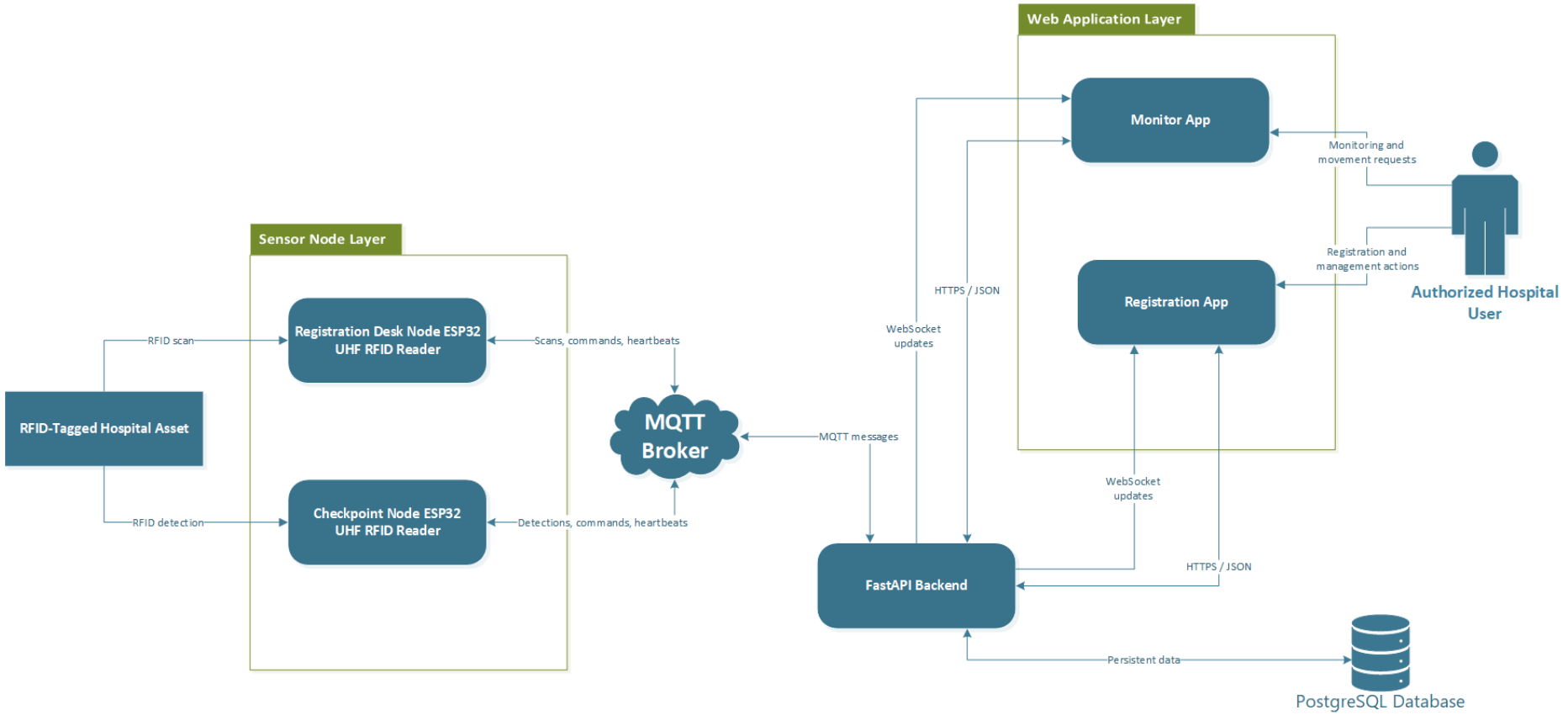

Figure 6. 9. 1 System communication overview and protocol boundaries.

The communication overview above shows that sensor nodes publish scans, detections, heartbeats, and acknowledgments through MQTT, while the backend applies business rules and stores accepted changes. The Registration App and Monitor App use HTTPS for requests and WebSocket for live notifications, without receiving raw MQTT access or broker credentials.

Table 6. 9. 1 Communication protocols, connected components, and purposes.

No.	Protocol	Used between	Purpose
1.	HTTPS/JSON	Frontend Apps ↔ FastAPI	Login, data loading, forms, commands, movement actions, user management, and settings.
2.	WebSocket/WSS	FastAPI → connected browsers	Live asset, node, movement, scan, alert, activity, and connection-state updates.
3.	MQTT	ESP32 nodes ↔ broker broker ↔ backend MQTT client	Registration scans, checkpoint detections, heartbeat, commands, and acknowledgments.
4.	UART/Serial	ESP32 ↔ UHF reader	Local reader communication and tag data acquisition.
5.	PostgreSQL connection	Backend ↔ database	Persistent storage and transactional queries.

6.10. Security Design Summary

Security is centered on the backend. Credentials are verified against hashed passwords, authenticated sessions use signed tokens, and protected routes apply role dependencies before executing service logic. Disabled accounts are rejected. Administrator safety policies prevent the final active administrator from being disabled, deleted, or demoted. Frontend route guards improve navigation but do not replace server-side enforcement.

Sensitive values such as database credentials, token secrets, broker credentials, and deployment addresses are stored in environment variables or protected configuration rather than committed source files. User-facing error messages avoid exposing stack traces, secrets, or internal database details. MQTT topic and payload validation prevents an arbitrary message from changing persistent state without passing backend checks.

6.11. Deployment Design Summary

In development, the backend, database, broker, frontends, and firmware can be run locally with environment-specific configuration. In the prototype deployment, the frontends are built as static React applications, the backend and PostgreSQL database are hosted on Render, and the deployed applications use HTTPS/WSS addresses. Sensor nodes require network access to the selected broker and backend-supported topic namespace.

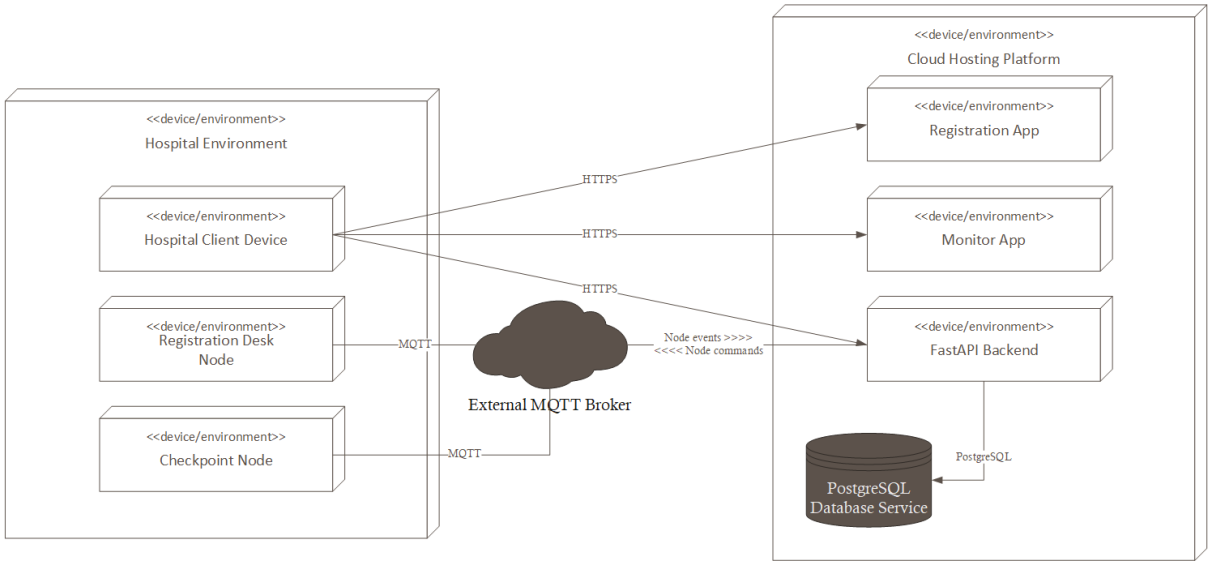

Figure 6. 11. 1 Prototype deployment architecture.

The deployment view above separates the hospital environment from the cloud-hosted prototype services. Nodes communicate with the external MQTT broker, browser clients reach the deployed applications and backend through HTTPS, and the backend connects to PostgreSQL. This diagram represents the prototype deployment and does not replace a production network-security or high-availability design.

7. System Implementation

This chapter explains how the architecture was realized in the current prototype. It focuses on responsibilities and important engineering choices rather than reproducing source code. File-level structure and class diagrams remain in the System Design Document and repositories.

7.1. Development Environment

Development used separate but integrated toolchains for firmware, backend services, two frontend applications, deployment, and diagnostics. The following table lists the principal technologies used to build, test, inspect, deploy, and document the prototype.

Table 7. 1. 1 Development tools and technologies by subsystem.

No.	Area	Tools and technologies
1.	Firmware	ESP32, PlatformIO, C++, UHF RFID reader, serial interface, native and embedded test environments.
2.	Backend	Python, FastAPI, SQLAlchemy, PostgreSQL, JWT authentication, MQTT client, WebSocket endpoints, pytest.
3.	Registration App	React, Vite, react-router-dom, modular CSS, REST services, WebSocket integration.
4.	Monitor App	React, Vite, role-protected routes, reusable tables/badges/hooks, REST and WebSocket integration.
5.	Deployment and inspection	Render, environment variables, Git, browser developer tools, Node-RED for MQTT inspection, PlantUML/diagram tooling.

7.2. Sensor-Node Firmware Implementation

The firmware uses shared modules so that the same codebase can support different node roles through configuration. Startup loads stored network and node information, enters setup mode when valid configuration is unavailable, obtains network access, connects to the MQTT broker, subscribes to supported commands, and begins recurring RFID and heartbeat work. The main loop remains non-blocking enough that connection recovery, RFID polling, status indication, and timers can coexist.

The wiring diagram below documents the main bench connections between the ESP32, UHF RFID reader, regulated breadboard power supply, status LEDs with resistors, and reset button. It is a logical implementation reference; final voltage, current capacity, grounding, enclosure, cable strain relief, and installation safety must still follow the component datasheets and hospital facilities requirements.

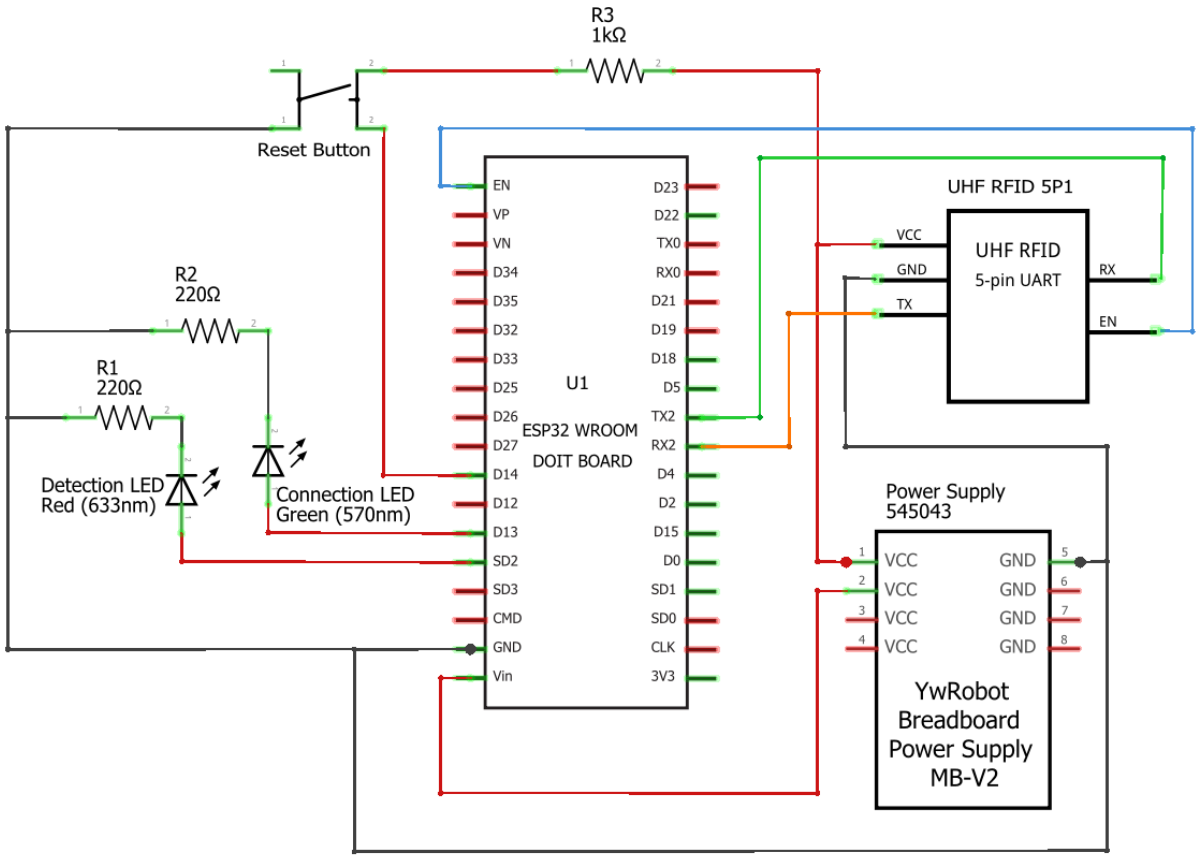

Figure 7. 2. 1 Sensor-node hardware wiring overview.

7.2.1. Wi-Fi Setup

A captive-portal workflow is available when the node lacks valid Wi-Fi configuration. The node exposes a local setup interface, validates submitted values, stores accepted configuration, and then attempts normal connection. During ordinary interruptions, the firmware retries the configured network rather than clearing settings or requiring manual re-provisioning.

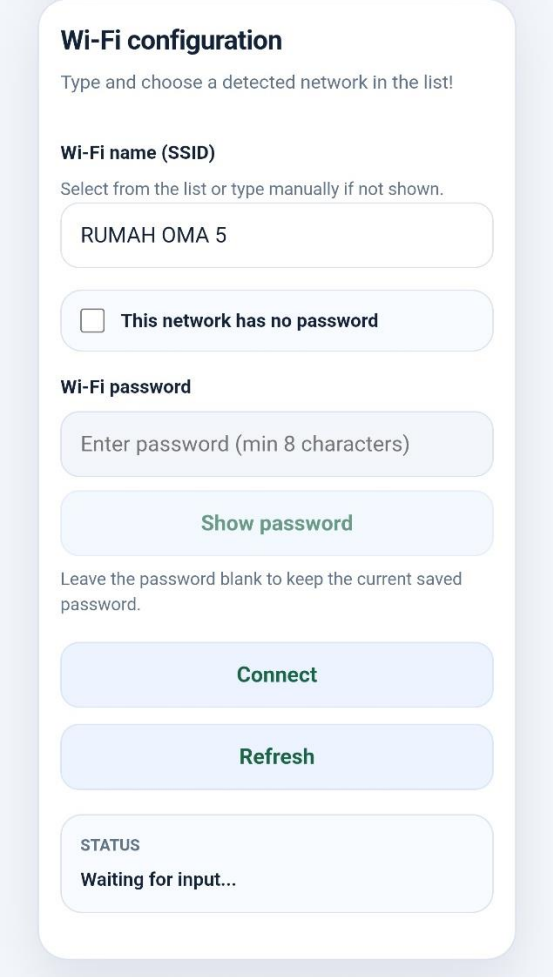

Figure 7. 2. 1. 1 Sensor-node captive-portal workflow: Wi-Fi configuration form.

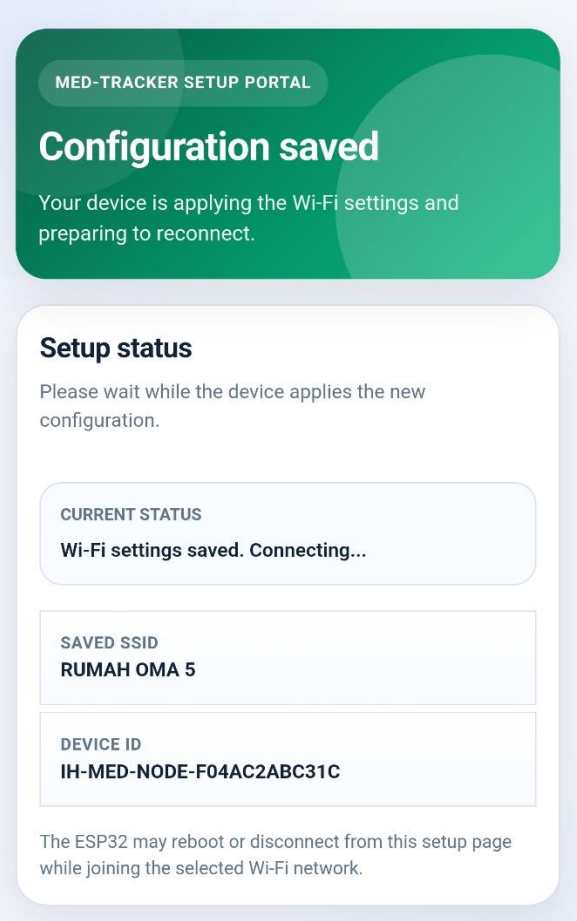

Figure 7. 2. 1. 2 Sensor node captive-portal workflow: Saved-configuration confirmation.

7.2.2. MQTT Communication

The MQTT module builds topics from the configured hospital and node identity, publishes registration scans, checkpoint detections, heartbeat status, and command acknowledgments, and subscribes to node-specific command topics. Reconnection includes resubscription so that commands such as physical blink continue to work after broker interruption. Payload builders centralize required fields and message type naming.

7.2.3. RFID Reading

The UHF reader is accessed through a dedicated RFID boundary. Tag identifiers are normalized before publication. A cooldown or suppression rule reduces repeated messages for the same tag within a short interval. The firmware does not decide whether a tag belongs to an asset or whether a movement is authorized; it reports the observation to the backend.

7.2.4. Heartbeat and Node Status

Nodes publish heartbeat messages at a configured interval. The backend records the latest valid heartbeat and presents the node as online. A background service compares the elapsed time with the offline timeout and changes eligible nodes to OFFLINE when the threshold is exceeded. Controlled dummy nodes used in simulation can follow a separate policy so they do not invalidate production timeout behavior.

7.2.5. LED and Blink Commands

LED behavior indicates setup, connection, operational, and error states. An authorized user can request a blink for an assigned, reachable node through the Registration App. The backend publishes the command, firmware temporarily overrides the normal LED pattern, and an acknowledgment reports whether the command was processed. This helps technicians match a physical device to its web record.

7.3. Backend Implementation

The FastAPI backend contains the product's authoritative business logic. HTTP routes validate authentication and request structure, then call service functions. MQTT handlers validate topic and payload information before calling the same domain services used by API workflows. SQLAlchemy models and transactions preserve persistent state, and WebSocket messages are produced after successful changes.

7.3.1. Authentication and Authorization

Login verifies the account, password hash, active state, and application access before returning a signed token and role information. Protected endpoints use reusable authentication and role dependencies. The backend also implements first-administrator setup and safety rules that prevent the final active administrator from being disabled, deleted, or demoted. Both frontends clear invalid sessions when a token expires or the backend returns an access denial.

7.3.2. Node Management

Node services accept discovery or first-contact data, preserve a stable node ID, validate assignment fields, and maintain alias, role, hospital, room, status, and last-ping information. Checkpoint Nodes require a valid location, while Registration Desk Nodes follow their own room rules. Blink commands are blocked for ineligible or offline nodes. Unassignment, disable/enable, and deletion are subject to reference and safety checks.

7.3.3. Asset Registration

A registration scan arrives through MQTT and is broadcast to eligible Registration App clients. The user provides the asset name and assigned room, then submits the registration request. The backend validates role, tag format, uniqueness, selected node eligibility, and required data before creating the asset. A newly registered asset remains "on the move" or pending placement until it is detected at its assigned checkpoint. Deregistration deactivates the asset according to the implemented rules and releases the tag from active use.

7.3.4. Asset Movement

Monitoring users create a movement request for a valid destination. The backend rejects duplicate pending requests and records the requester and timestamps. Registration-side users approve or reject the request. Approval authorizes movement but does not change the assigned room immediately. The asset remains in transit until a checkpoint at the approved destination detects it. A different-room detection can produce an unauthorized state and warning instead of completing the request.

7.3.5. MQTT Processing

The backend MQTT worker subscribes to hospital-scoped topics, validates message categories, resolves the node identity, and routes scans, detections, heartbeats, and acknowledgments to dedicated handlers. Invalid topic structures, unknown nodes, malformed payloads, and unsupported message types are rejected without committing business changes. Backend-to-node commands use targeted topics rather than broad broadcast whenever possible.

7.3.6. WebSocket Broadcasting

Connected clients receive typed events such as asset updates, node updates, movement changes, alerts, activity records, and registration scans. The event tells the frontend which collection or workflow changed, while the API remains available for full refresh. Socket authentication and reconnection behavior prevent anonymous or expired sessions from repeatedly opening unauthorized connections.

7.3.7. Database Persistence

PostgreSQL stores users, nodes, assets, movement requests, detections, alerts, activities, and settings. Database work is performed through SQLAlchemy sessions. Service operations validate state before commit and roll back when a business rule or unexpected exception prevents completion. Unique constraints and service-level checks protect active tag identity and movement consistency.

7.4. Registration App Implementation

The Registration App provides a role-aware shell and protected pages. Node Registration uses a toolbar, assignment/edit form, responsive table, status badges, search, sorting, pagination, and blink feedback. Asset Registration combines scanned-tag state, register/deregister forms, asset table, and movement-request decision cards. User Management and Account Security provide administrative and self-service account workflows, while the settings page manages allowed configuration values where enabled.

Implementation improvements include modular hooks and services, consistent status normalization, sticky headers and action columns, loading and connection feedback, duplicate-submission guards, modal keyboard behavior, and compact layouts for half-window and mobile use. Backend errors are displayed through user-safe banners or field feedback.

The Registration Dashboard presents a role-aware overview of the modules available to the authenticated user. The upper area provides system counts and quick-access cards, while the lower area explains the user's permitted functions and highlights operational items that may require attention. The screenshot is divided into two segments so the full page remains readable in the report.

The Node Registration page combines operational status with technical management actions. Users can review discovered and assigned nodes, search the list, compare role, hospital, room, and last-ping data, and perform eligible actions such as blink identification or editing. Status badges and the connection indicator distinguish physical node availability from browser connectivity.

The Asset Registration page combines RFID scan feedback, registration and deregistration controls, pending movement decisions, and the registered asset table. The interface shows tag identity, asset status, last known location, latest time information, and available actions so registration staff can manage the complete asset lifecycle without directly interacting with MQTT or database records.

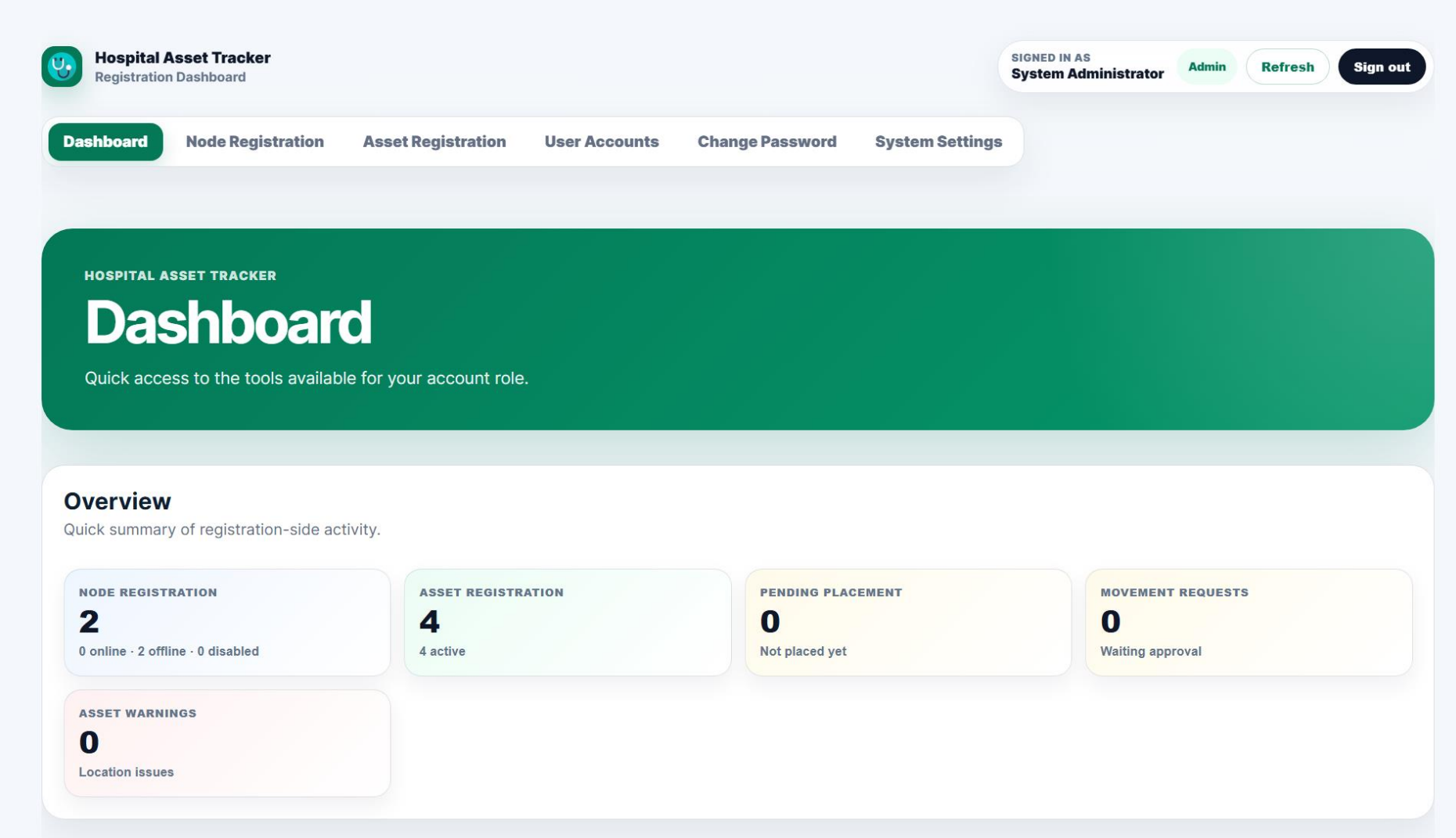

Figure 7. 4. 1 Registration App dashboard showing role-aware summaries and access overview.

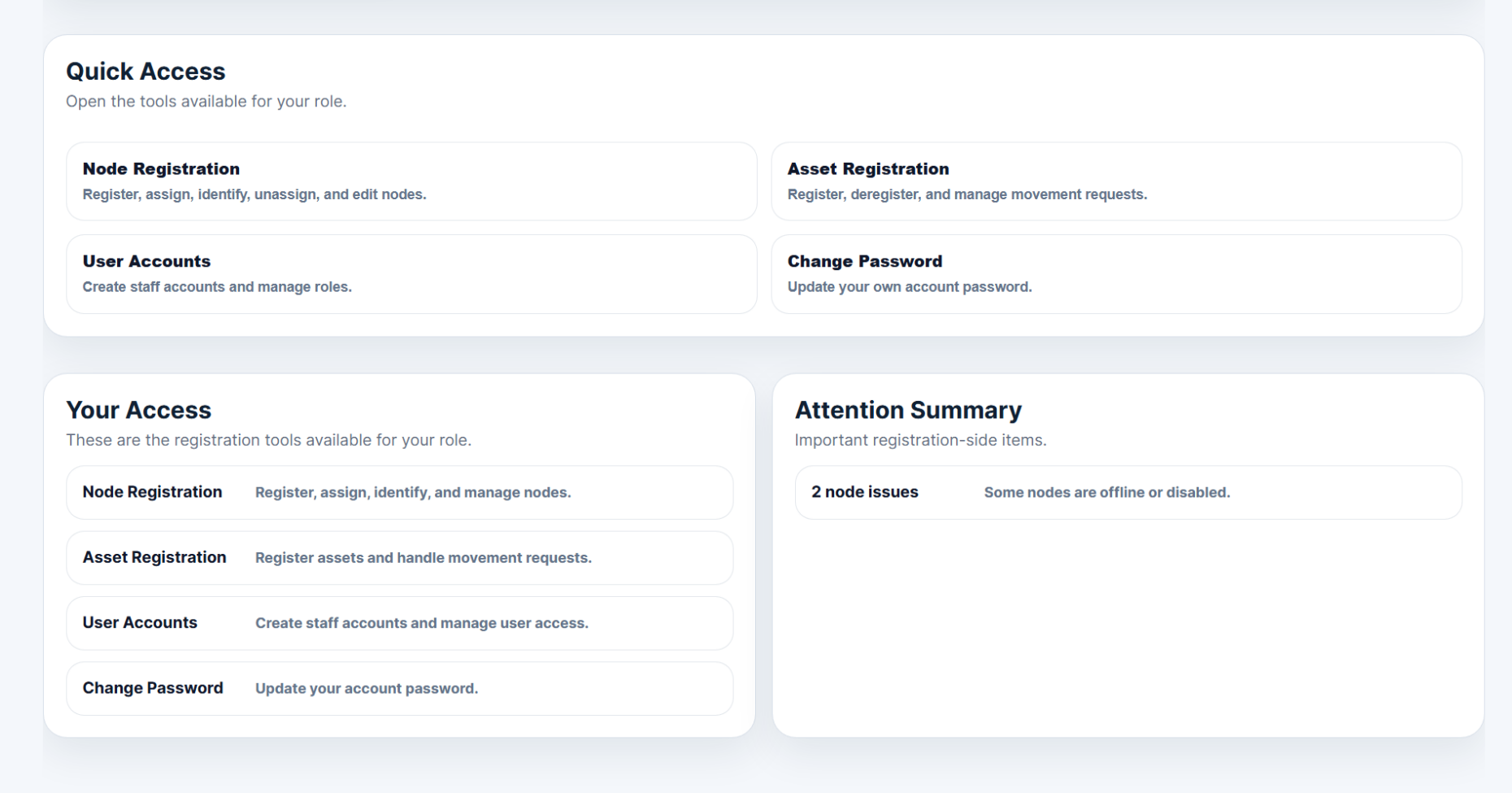

Figure 7. 4. 2 Registration App dashboard showing quick access and attention indicators.

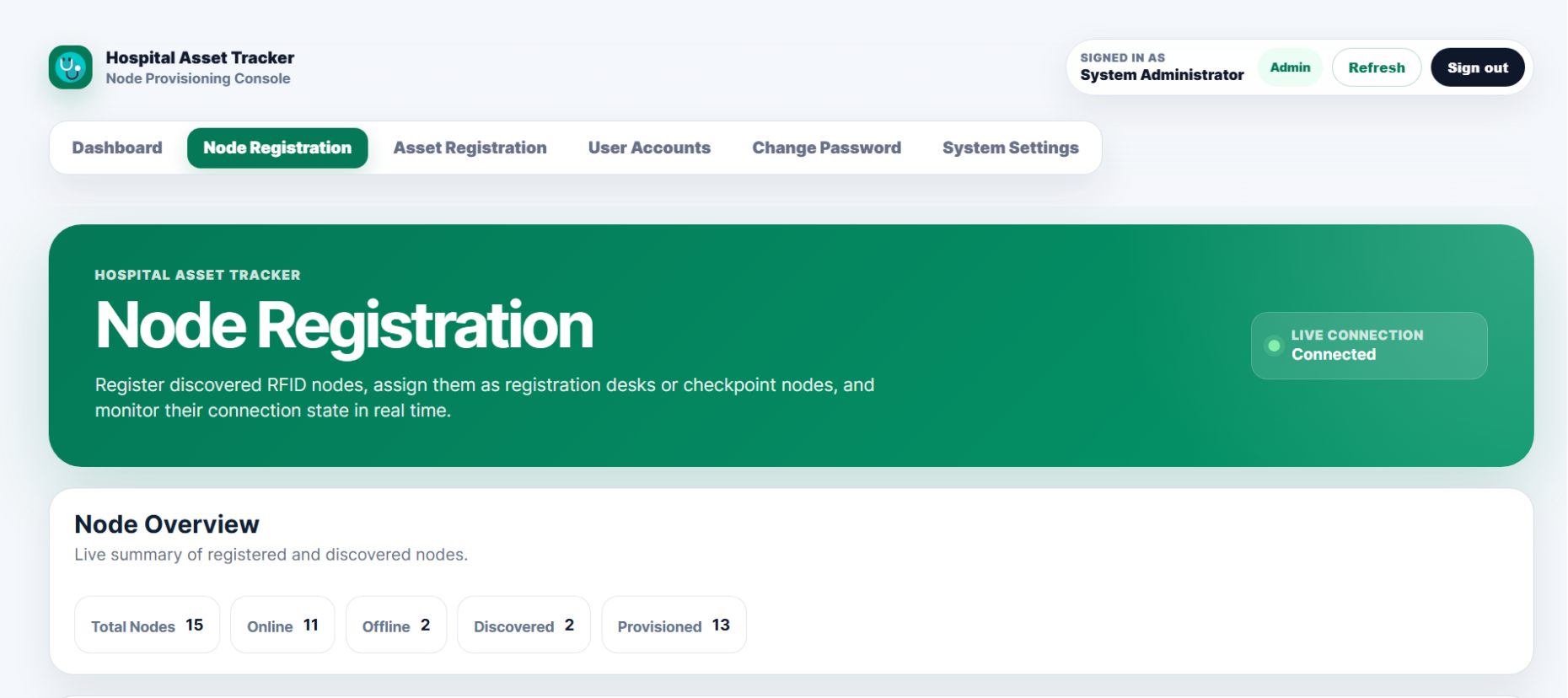

Figure 7. 4. 3 Node registration page, with node overviews.

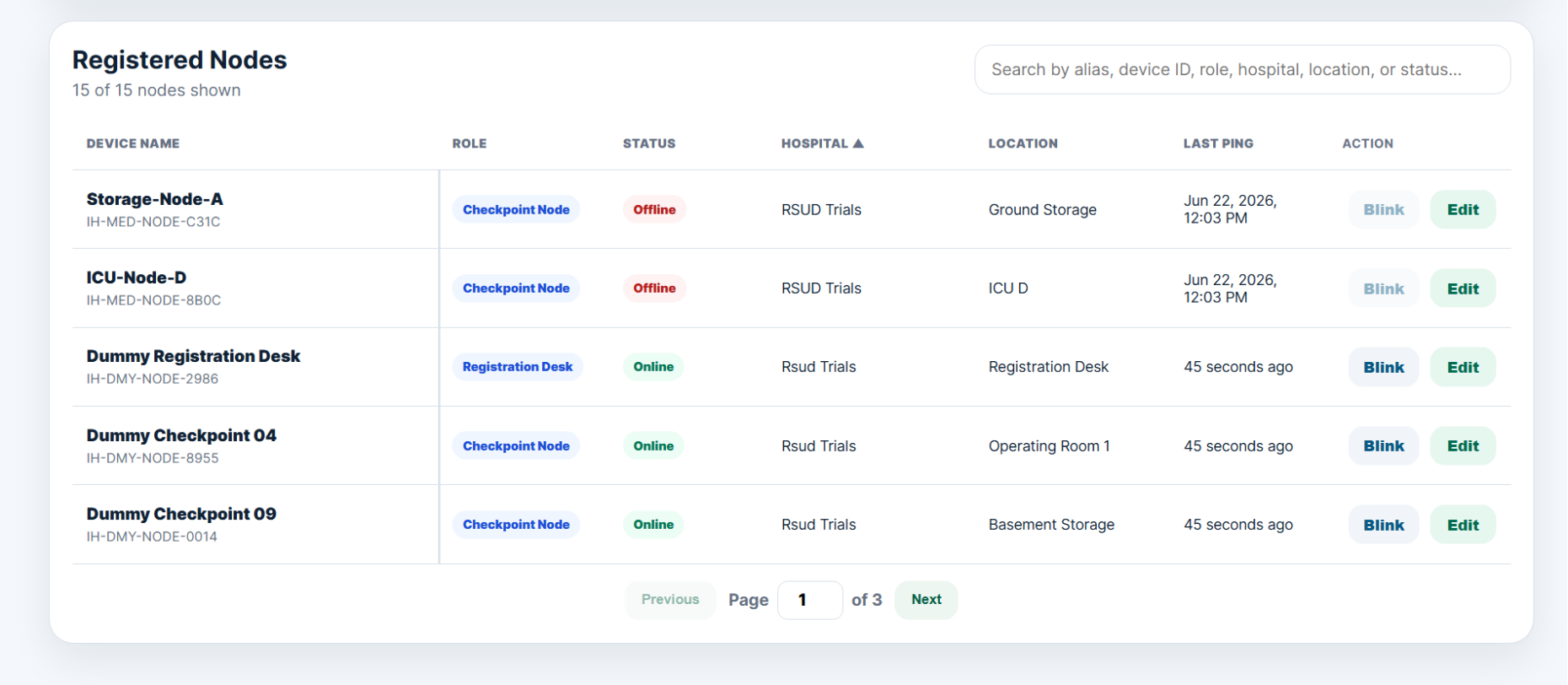

Figure 7. 4. 4 Node Registration page showing node status, location, search, pagination, and management actions.

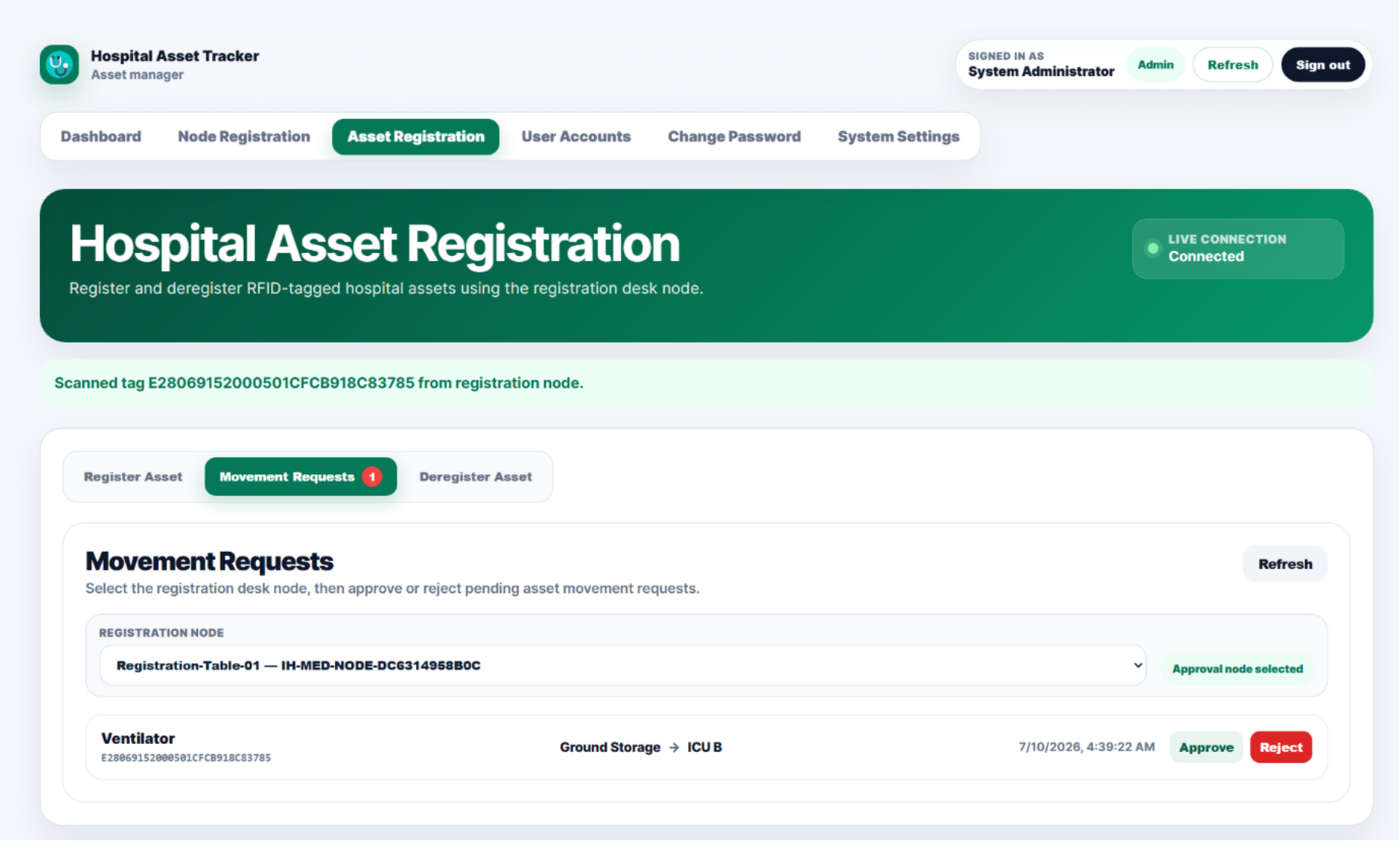

Figure 7. 4. 5 Asset Registration page showing scanned-tag feedback and movement request approval choices.

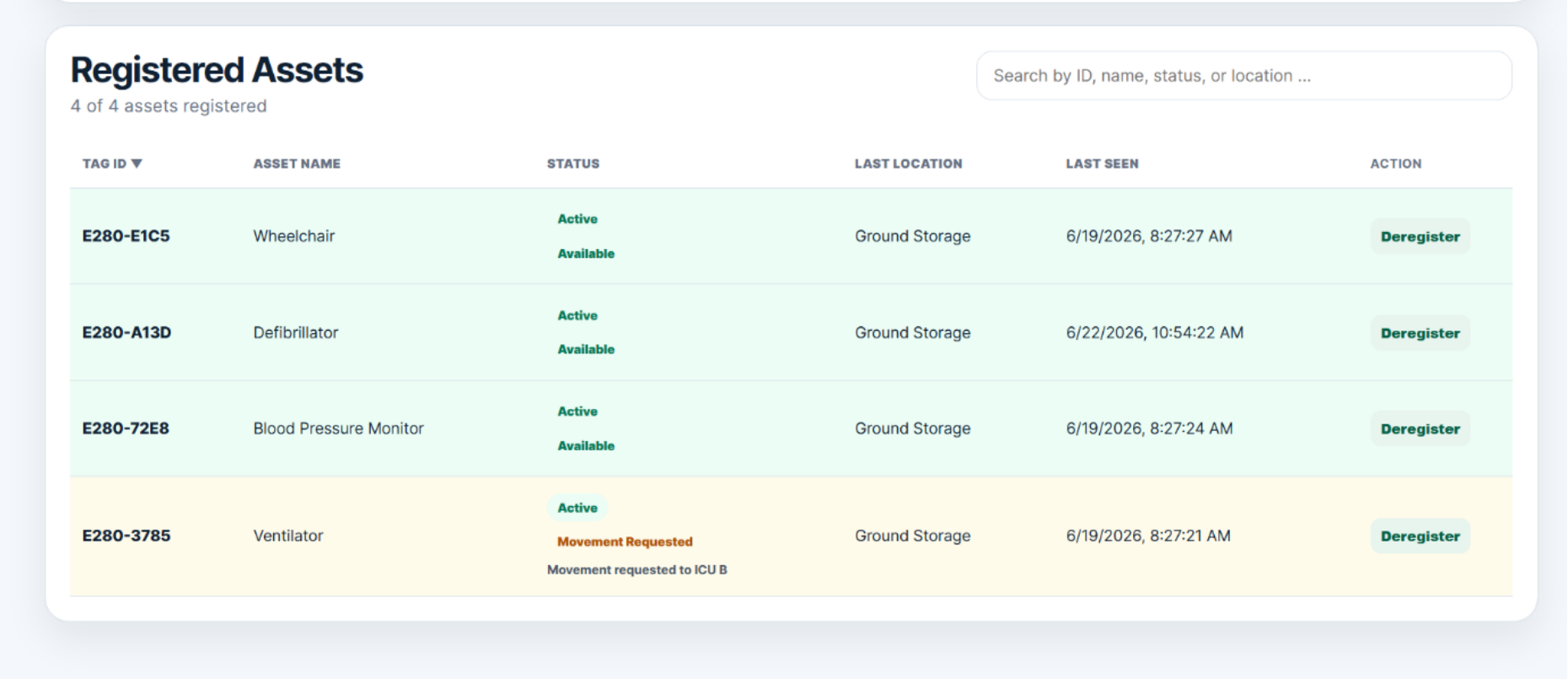

Figure 7. 4. 6 Asset Registration page showing registered asset records and their status.

7.5. Monitor App Implementation

The Monitor App uses a shared top bar and module navigation across Dashboard, Assets, Nodes, Alerts, and Activity. The dashboard derives summary cards from the same status rules used by the detailed pages. The Assets page displays tag, name, assigned room, current room, last seen, and flow status, with movement request or cancel actions according to role and request state. The Nodes page presents role, location, heartbeat, and normalized operational status.

The Monitor Dashboard provides an operational summary rather than administrative controls. It combines counts for assets, nodes, movement states, and warnings with recent warning and activity panels. The screenshot is divided into three segments to preserve readability while showing the complete dashboard.

The Asset Monitor table presents the current operational view of each registered asset. It compares current and assigned locations, displays the flow state and latest observation, and exposes a movement-request action only when the user role and asset state permit it. This layout supports rapid comparison without requiring users to understand the underlying event-processing logic.

The Alert Center groups user-facing warnings by severity, type, status, location, message, and time. Search and filters help monitoring staff focus on conditions such as offline nodes, unknown tags, or unauthorized movement. Alerts complement the activity history by emphasizing situations that require attention rather than every recorded event. This feature prioritizes actionable warnings such as unknown tags, offline nodes, and unauthorized movement. Activity records important state changes while suppressing repetitive heartbeat noise from the user-facing feed. Search, sorting, pagination, status badges, loading indicators, and responsive table behavior are implemented as reusable UI patterns.

The image below shows what the Monitor web app's top bar looks like. There, users can access the dashboard, assets, nodes, alerts, and activities pages, as mentioned before.

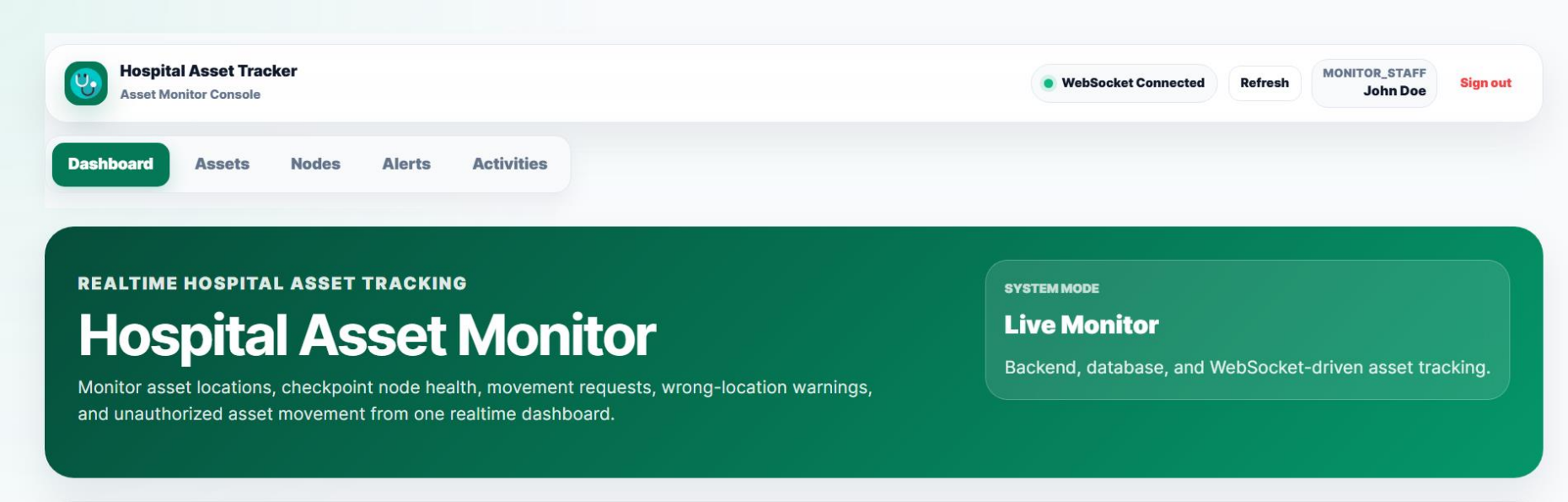

Figure 7. 5. 1 Monitor app dashboard top bar, with tab shortcuts to dashboard, asset, node, alerts, and activities monitor page.

The image below is the Monitor App Dashboard's live overview. Here, users can quickly see almost every condition of the nodes attached to the same network as the app, as well as all registered assets within it. Each box can be clicked for quick access to the page within the app related to the instance shown inside each box.

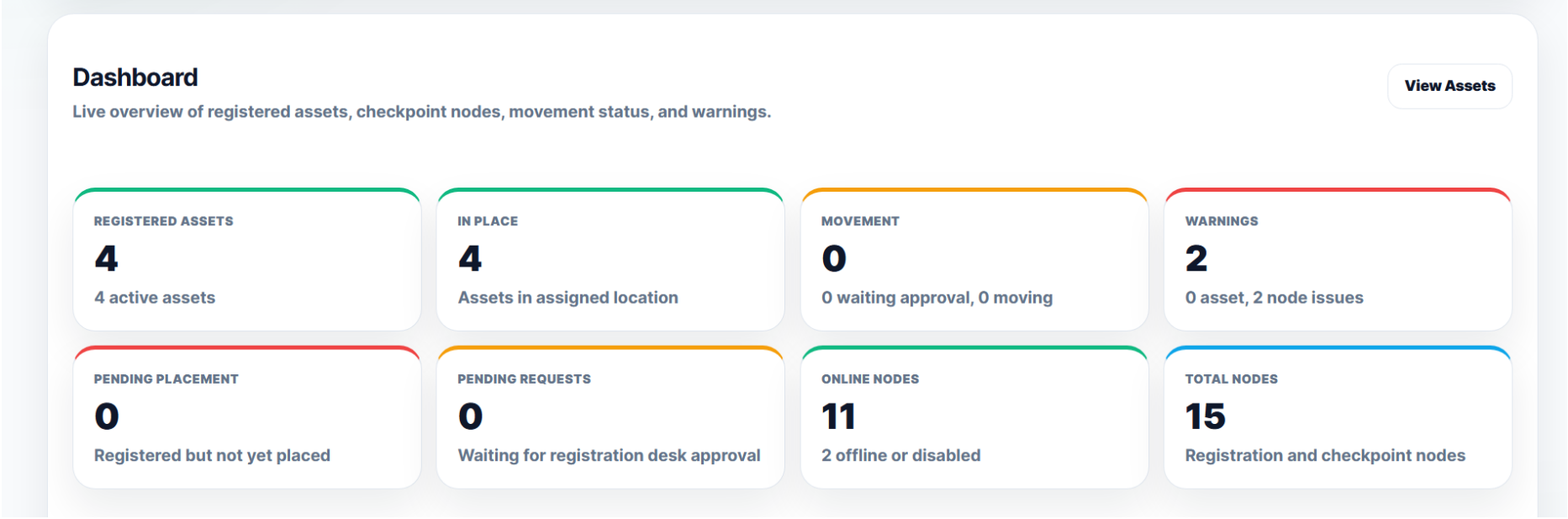

Figure 7. 5. 2 Monitor app dashboard live overview section.

The image below shows the quick access to alerts and activities, with a summary of instances within each app page.

As mentioned before, the app shall show a warning if any asset is moved without any approval, and if any node gets disconnected from the server for more than three minutes. The Recent Warnings box on the left side of the image below shows that warning to the user briefly on the dashboard.

Other than that, the Recent Activity box on the right side shows users what happened recently with the system, nodes, and assets.

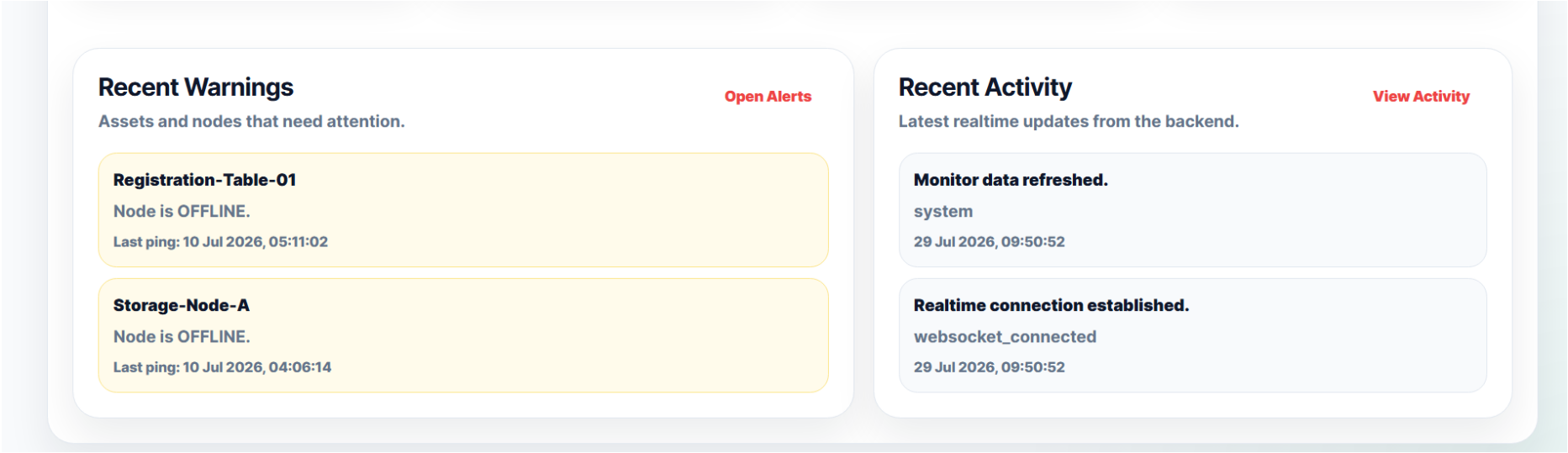

Figure 7. 5. 3 Monitor app dashboard recent warning and activities log summary.

The image below shows how the asset monitor page indicates whether an asset was moved without approval from the registration desk.

If an asset's current place is not the same as the assigned place, the asset status is labeled as an unauthorized movement. If the asset's expected place is mentioned, that means the movement request was approved, and the asset is on the move. Otherwise, if the asset's current and assigned place are identical, that means the asset is in place.

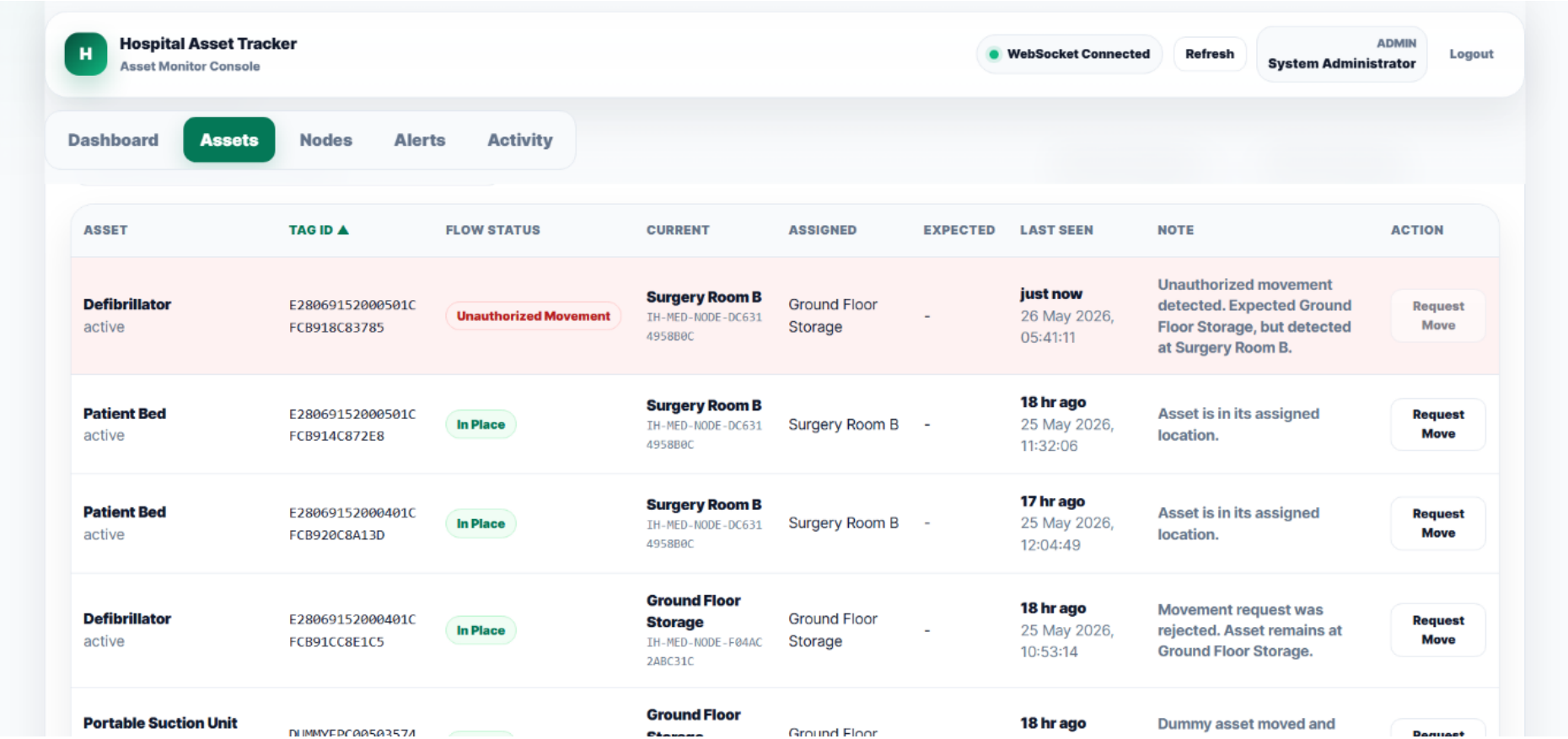

Figure 7. 5. 4 Asset Monitor page showing current, assigned, and expected locations with flow-state indicators.

The image below is the alert center page of the monitor app. It shows what the alert is, which type of system it is, along with its severity, location, and event time as evidence. In addition, the page shows a short message note about what happened with the subject in the alerts.

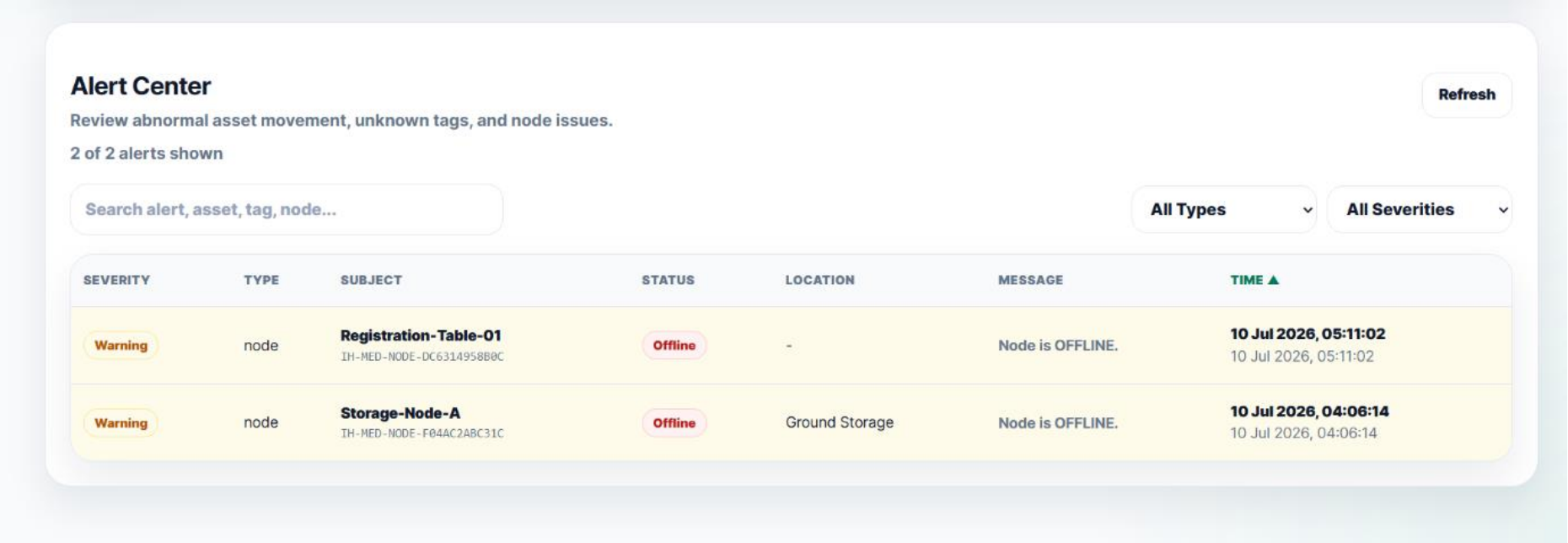

Figure 7. 5. 5 Monitor App Alert Center showing alert severity, type, subject, status, location, message, and event time.

7.6. Deployment and Configuration

Development builds use local API and WebSocket addresses, while production builds use environment-specific HTTPS and WSS values. The backend receives database, JWT, MQTT, hospital, and deployment configuration from environment variables or protected settings. Secret files are excluded from Git, and previously tracked environment files must be removed from repository history or index when necessary.

The deployed prototype uses Render for the backend and PostgreSQL service. Because free databases or services can expire, the project requires regular export and a documented migration path. Production deployment should also use a broker configuration that isolates hospitals, authenticates nodes, and encrypts traffic where supported.

7.7. Important Implementation Challenges

Several problems became visible only when complete workflows were exercised across firmware, messaging, backend, database, and frontend boundaries. The table summarizes the most important implementation challenges, their user-visible effect, and the corrective response.

Table 7. 7. 1 Major implementation challenges, observed effects, and responses.

No.	Challenge	Observed effect	Implemented response
1.	Node online/offline inconsistency	One application showed ONLINE after the timeout or until refresh.	Centralized timeout logic, WebSocket node updates, normalized status display, and API refresh after reconnect.
2.	Movement workflow ambiguity	Approval could be mistaken for completed movement.	Separated PENDING, APPROVED, ON_THE_MOVE, COMPLETED, REJECTED, and UNAUTHORIZED behavior.
3.	Duplicate requests or submissions	Repeated clicks created conflicting or confusing state.	Backend duplicate checks and frontend submission guards/disabled states.
4.	Session persistence and role routing	Users were logged out after refresh or saw incorrect access restrictions.	Shared session helpers, token validation, role-aware routes, and backend dependency review.
5.	Large source files	Features became difficult to test and maintain.	Split pages, hooks, services, utilities, repositories, and handlers while preserving behavior.
6.	Responsive data-heavy screens	Tables, toolbars, and modals were difficult in half-window/mobile layouts.	Sticky columns/headers, compact top bars, responsive forms, controlled overflow, and mobile reflow.
7.	Multi-hospital MQTT configuration	Hardcoded topics risked mixing node data.	Hospital-scoped topic namespace and Admin/Test User-managed settings design.

8. Testing and Validation Summary

Testing was performed at firmware, backend, frontend, integration, and system levels. The System Requirement Document defines AC-001 to AC-015 as the acceptance baseline, while the Requirements and Use Case Specification supplies the normal and alternative workflows to exercise. Detailed test identifiers, environments, evidence, defects, and retest history should be maintained in the System Test Report. This chapter summarizes current evidence and clearly separates demonstrated behavior from criteria that remain partial or open.

8.1. Testing Objectives

Testing focused on protecting business rules, preserving consistent state across asynchronous communication, validating role access, and identifying limitations before production-readiness claims were made. These objectives guide the evidence summarized throughout this chapter.

Table 8. 1. 1 Testing objectives for the integrated prototype.

No.	Testing objectives
1.	Verify that business rules reject invalid or unauthorized operations without corrupting data.
2.	Verify that firmware modules continue operating through ordinary connection failures and repeated RFID observations.
3.	Verify that API, MQTT, database, and WebSocket layers produce consistent visible state.
4.	Verify role access, session expiry, password handling, and administrator safety rules.
5.	Verify movement requests, approvals, rejections, completion, and unauthorized movement transitions.

6.	Verify that the web applications remain usable at full-screen, half-window, and mobile widths.
7.	Identify residual defects and unperformed validation before claiming production readiness.

8.2. Testing Strategy

The strategy uses inspection, analysis, demonstration, automated unit tests, integration tests, end-to-end tests, and user acceptance, matching the verification approach defined by the System Requirement Document. Unit tests isolate business rules and firmware helpers. Simulation uses dummy nodes or assets to reproduce status and movement scenarios. Integration tests verify communication across subsystem boundaries. End-to-end checks follow the normal and alternative paths of the relevant use cases. Security tests focus on authentication, authorization, unsafe input, and secret handling. Formal hospital acceptance is treated as a separate activity.

8.3. Unit Testing Summary

Backend tests use pytest to verify service rules, route permissions, authentication dependencies, administrator safety, node assignment, asset registration, movement state changes, and error handling. A shared test configuration and fixtures reduce duplication. Tests were refactored alongside service modules so route handlers do not need to contain database business logic.

Firmware tests use PlatformIO native and ESP32 environments for helpers, guards, configuration, payload construction, reconnection behavior, and module boundaries where hardware can be abstracted. Hardware-dependent reader, Wi-Fi, and timing behavior still requires embedded or bench verification. The frontend applications have primarily been validated through targeted manual functional and responsive tests; automated component and browser tests remain an area for improvement.

The backend test output provides retained evidence that the automated pytest suite executed service, route, authorization, node, asset, movement, and policy tests successfully in the captured run. The screenshot is divided into segments for readability. It supports the unit-testing summary but does not by itself prove complete acceptance-criterion or production-environment coverage.

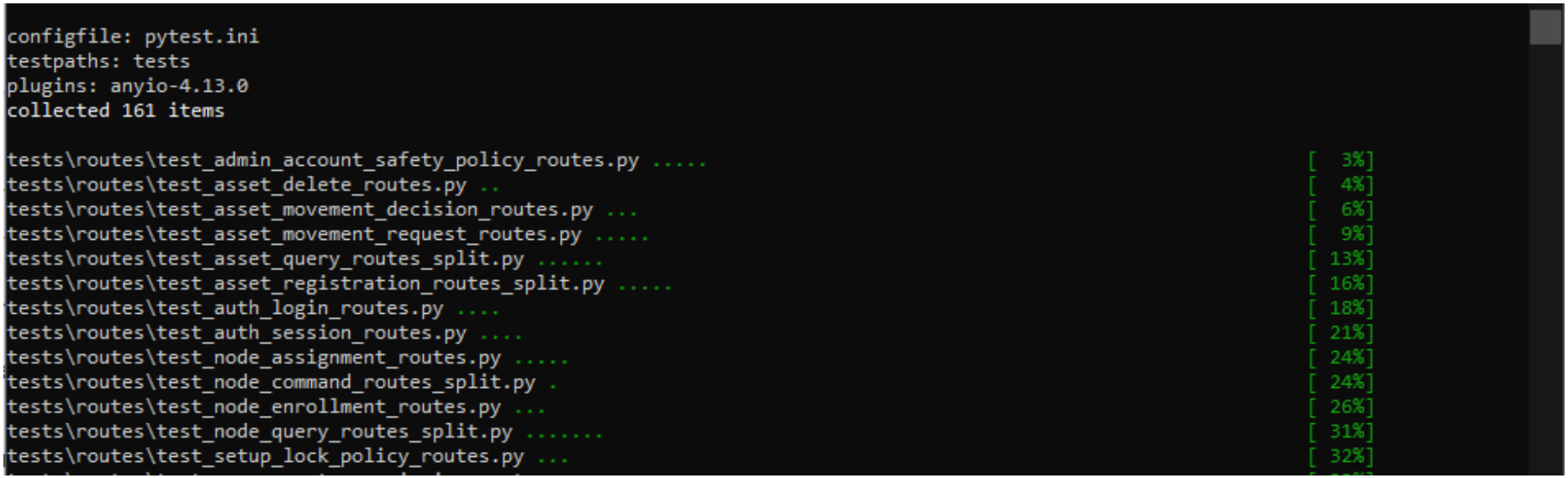

Figure 8. 3. 1 Backend pytest execution, testing admin safety policy, asset and node behaviors, account role authority access, until the test lock policy.

```
tests\routes\test_user_create_permission_routes.py .                                [ 33%]
tests\routes\test_user_create_routes_split.py ....                                  [ 36%]
tests\routes\test_user_password_reset_routes.py .                                   [ 36%]
tests\routes\test_user_password_routes.py ....                                      [ 39%]
tests\routes\test_user_query_routes.py ....                                         [ 41%]
tests\routes\test_user_update_permission_routes.py ..                               [ 42%]
tests\routes\test_user_update_routes_split.py ...                                   [ 44%]
tests\services\assets\test_asset_deregister_service.py ..                           [ 45%]
tests\services\assets\test_asset_location_placement_service.py .                    [ 46%]
tests\services\assets\test_asset_location_timestamp_service.py ...                  [ 48%]
tests\services\assets\test_asset_location_update_service.py ...                     [ 50%]
tests\services\assets\test_asset_movement_cancel_service.py ..                      [ 51%]
tests\services\assets\test_asset_movement_create_service.py .....                   [ 54%]
tests\services\assets\test_asset_movement_decision_service.py ...                   [ 56%]
tests\services\assets\test_asset_register_location_validation_service.py ..         [ 57%]
tests\services\assets\test_asset_register_success_service.py .                      [ 58%]
tests\services\assets\test_asset_register_validation_service.py .....               [ 61%]
```

Figure 8. 3. 2 Backend pytest execution, user roles and permissions, and asset positioning.

```
tests\services\nodes\test_node_heartbeat_service.py ....                            [ 63%]
tests\services\nodes\test_node_offline_monitor_service.py ...                       [ 65%]
tests\services\test_node_service.py .........                                       [ 71%]
tests\services\users\test_user_active_policy_service.py ....                       [ 73%]
tests\services\users\test_user_create_query_service.py ....                        [ 76%]
tests\services\users\test_user_password_service.py ..                              [ 77%]
tests\services\users\test_user_update_service.py ....                              [ 80%]
tests\services\users\test_user_validation_service.py .....                         [ 83%]
tests\unit\test_asset_flow_initial_placement.py ..                                  [ 84%]
tests\unit\test_asset_flow_movement_states.py ...                                   [ 86%]
tests\unit\test_location_matcher.py .......                                         [ 90%]
tests\unit\test_mqtt_topics.py .........                                            [ 96%]
tests\unit\test_node_reference_service.py ......                                    [100%]

============================================== 161 passed in 33.09s ==============================================
```

Figure 8. 3. 3 Backend pytest execution showing 161 passed tests in the captured run.

The first firmware test output shows PlatformIO native tests for configuration parsing, command and payload validation, timing decisions, and related firmware logic that can be isolated from physical hardware. Splitting the terminal output across four segments keeps the individual test names and pass results legible.

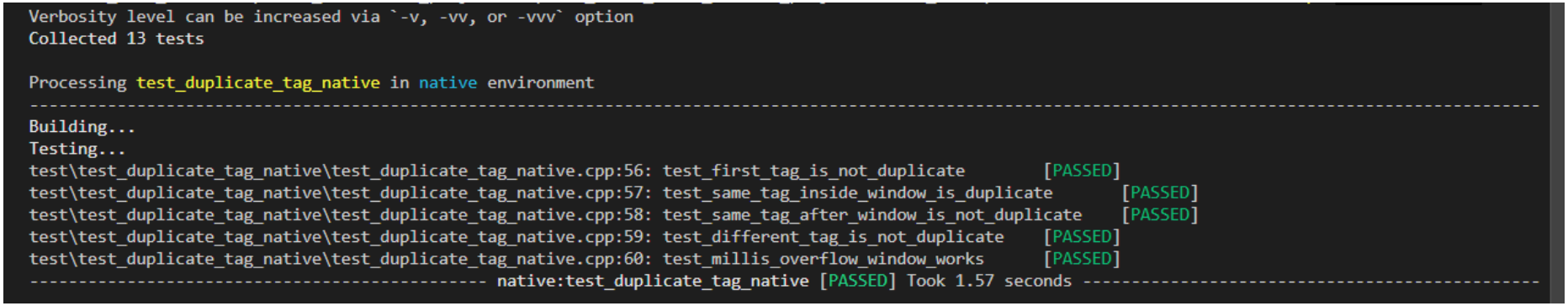

Figure 8. 3. 4 Firmware PlatformIO native duplicate tag test.

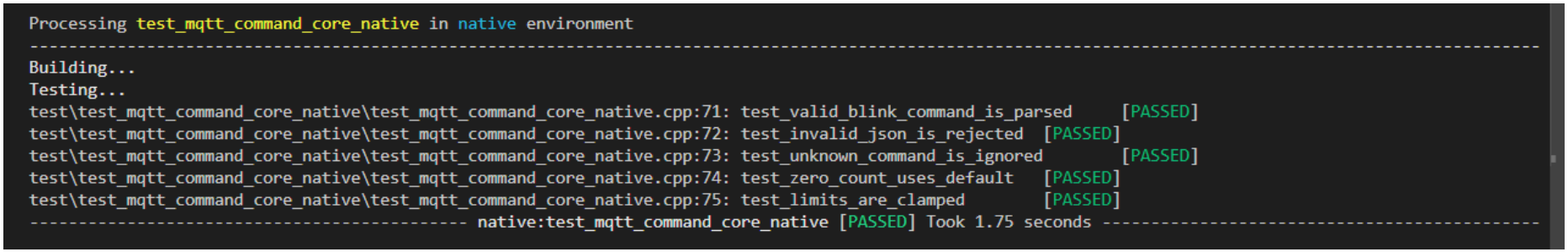

Figure 8. 3. 5 Firmware PlatformIO native MQTT command test.

```
Processing test_remote_config_validator_native in native environment
-------------------------------------------------------------------------------------------------------------------------------------------------------
Building...
Testing...
test\test_remote_config_validator_native\test_remote_config_validator_native.cpp:69: test_valid_checkpoint_config_is_accepted   [PASSED]
test\test_remote_config_validator_native\test_remote_config_validator_native.cpp:70: test_valid_registration_config_allows_empty_room   [PASSED]
test\test_remote_config_validator_native\test_remote_config_validator_native.cpp:71: test_checkpoint_config_requires_room       [PASSED]
test\test_remote_config_validator_native\test_remote_config_validator_native.cpp:72: test_config_rejects_missing_mqtt_host      [PASSED]
test\test_remote_config_validator_native\test_remote_config_validator_native.cpp:73: test_config_rejects_unprovisioned_response [PASSED]
------------------------------------------- native:test_remote_config_validator_native [PASSED] Took 1.49 seconds -------------------------------------------
```

Figure 8. 3. 6 Firmware PlatformIO native remote config validator test.

```
Processing test_state_decision_native in native environment
-------------------------------------------------------------------------------------------------------------------------------------------------------
Building...
Testing...
test\test_state_decision_native\test_state_decision_native.cpp:44: test_wifi_disconnected_tries_wifi    [PASSED]
test\test_state_decision_native\test_state_decision_native.cpp:45: test_unprovisioned_tries_provisioning        [PASSED]
test\test_state_decision_native\test_state_decision_native.cpp:46: test_mqtt_disconnected_tries_mqtt    [PASSED]
test\test_state_decision_native\test_state_decision_native.cpp:47: test_heartbeat_due_publishes_heartbeat       [PASSED]
test\test_state_decision_native\test_state_decision_native.cpp:48: test_ready_node_reads_rfid   [PASSED]
---------------------------------------------- native:test_state_decision_native [PASSED] Took 1.55 seconds ----------------------------------------------
```

Figure 8. 3. 7 Firmware PlatformIO native state decision test.

The second firmware test output continues the PlatformIO results and summarizes the executed native environments. These tests increase confidence in deterministic helper and state logic, while Wi-Fi behavior, UHF reader performance, electrical stability, and real MQTT connectivity still require embedded bench and integration testing.

```
Processing test_text_utils_native in native environment
-------------------------------------------------------------------------------------------------------------------------------------------------------
Building...
Testing...
test\test_text_utils_native\test_text_utils_native.cpp:47: test_hospital_name_becomes_slug      [PASSED]
test\test_text_utils_native\test_text_utils_native.cpp:48: test_extra_symbols_are_cleaned       [PASSED]
test\test_text_utils_native\test_text_utils_native.cpp:49: test_empty_name_uses_fallback        [PASSED]
test\test_text_utils_native\test_text_utils_native.cpp:50: test_small_buffer_is_rejected        [PASSED]
---------------------------------------------- native:test_text_utils_native [PASSED] Took 1.77 seconds ----------------------------------------------
```

Figure 8. 3. 8 Firmware PlatformIO native utility test.

```
Processing test_timing_native in native environment
-------------------------------------------------------------------------------------------------------------------------------------------------------
Building...
Testing...
test\test_timing_native\test_timing_native.cpp:37: test_interval_is_due_at_exact_time   [PASSED]
test\test_timing_native\test_timing_native.cpp:38: test_interval_is_not_due_before_time [PASSED]
test\test_timing_native\test_timing_native.cpp:39: test_interval_is_due_after_time      [PASSED]
test\test_timing_native\test_timing_native.cpp:40: test_interval_handles_millis_overflow       [PASSED]
---------------------------------------------- native:test_timing_native [PASSED] Took 1.94 seconds ----------------------------------------------
```

Figure 8. 3. 9 Firmware PlatformIO native timing interval test.

```
============================================================================ SUMMARY ============================================================================
Environment    Test                                 Status    Duration
-------------  -----------------------------------  --------  ------------
native         test_duplicate_tag_native            PASSED    00:00:01.571
native         test_mqtt_command_core_native        PASSED    00:00:01.755
native         test_remote_config_validator_native  PASSED    00:00:01.489
native         test_state_decision_native           PASSED    00:00:01.548
native         test_text_utils_native               PASSED    00:00:01.769
native         test_timing_native                   PASSED    00:00:01.939
================================================== 28 test cases: 28 succeeded in 00:00:10.071 ==================================================
PS D:\ic_tech_course\imperial_healthtech_project\hospital_fixed_asset_tracker_project\source_code\products\AssetTrackerSensorNode>
```

Figure 8. 3. 10 Firmware PlatformIO native test summary showing 28 successful test cases across six test environments.

8.4. Simulation Testing Summary

Dummy assets and nodes were used to create repeatable movement, unauthorized-location, and online/offline scenarios without requiring every physical checkpoint to be available. Simulation helped expose differences between backend status and frontend display, including dummy nodes being marked offline by the normal timeout and inconsistent state across applications. The resulting policy separates controlled simulation behavior from normal heartbeat-based production behavior.

8.5. Integration Testing Summary

Integration checks covered MQTT message intake, backend validation, database update, and WebSocket propagation. Node blink was verified as a complete command path:

Registration App request → backend authorization → MQTT command → firmware LED behavior → acknowledgment → backend update → frontend feedback

RFID scan and checkpoint workflows were checked through the same multi-component path.

The project also validated frontend-to-backend authentication, protected routes, movement actions, and live state changes. Remaining integration work includes broader failure injection, broker TLS and credential tests, database migration drills, and multi-hospital isolation using more than one realistic namespace and node group.

8.6. System and End-to-End Testing Summary

End-to-end scenarios included login, node discovery and assignment, asset registration from a scanned tag, initial placement at the assigned room, movement request, approval, destination detection, movement completion, unauthorized movement, offline-node detection, and session recovery. These checks demonstrate that the prototype's central workflows are coherent across hardware, messaging, backend, persistence, and UI.

Not every test has yet been executed as a formal repeatable case with captured evidence. The final System Test Report should provide the exact environment, test data, expected result, actual result, screenshots or logs, defect reference, and retest status for each acceptance criterion.

Table 8. 6. 1 Acceptance-criterion groups, current assessment, and evidence still required.

No.	Acceptance criterion group	Related criteria	Current report assessment	Evidence still required
1.	Initialization and account protection	AC-001 to AC-002	Core first-admin, authentication, password, role, and administrator-safety behavior has implementation and development-test evidence.	Final repeatable cases with environment, actual results, and retest status.
2.	Node lifecycle and blink	AC-003 to AC-004	Core node management and blink command path have been demonstrated.	Complete invalid-transition coverage and retained command/ACK evidence.
3.	Asset lifecycle and movement	AC-005 to AC-008	Registration, duplicate prevention, deregistration, request decisions, destination completion, and wrong-location behavior are implemented for representative scenarios.	Formal evidence for every normal and alternative use-case path.
4.	Warnings and live updates	AC-009 to AC-011	Offline, unknown-tag, unauthorized-movement, WebSocket reconnect, and live-update behavior have partial demonstration evidence.	Repeatable timeout, interruption, and recovery tests across both applications.

5.	Settings and responsive UI	AC-012 to AC-013	Role-aware settings and responsive layouts are implemented and manually reviewed.	Final role matrix, invalid-value cases, and recorded desktop/half-window/tablet/mobile results.
6.	Failure recovery and deployment	AC-014 to AC-015	Partially addressed through error handling, configuration separation, and deployment practice.	Structured failure injection, clean deployment checklist execution, migration/restore evidence, and secret audit.

8.7. Security Testing Summary

Security verification included invalid login behavior, disabled-account rejection, missing or expired token handling, role restrictions on protected endpoints, frontend redirect after access denial, password hashing, and the policy protecting the last active administrator. Attempts to submit script-like or XSS-style input were used to check that unsafe values were not executed in the browser, and that backend validation or output rendering remained safe.

The prototype has not completed an independent penetration test, formal threat model review, dependency vulnerability process, broker certificate validation, or production secrets audit. These activities remain necessary before handling real hospital operational data.

8.8. User Acceptance Testing Summary

The interface and workflows have received iterative project review and demonstration feedback, particularly for table usability, responsive behavior, role access, movement actions, and status clarity. However, a formal user-acceptance test with representative hospital staff, signed acceptance criteria, and measured task outcomes has not yet been completed. This report therefore treats usability conclusions as prototype evaluation rather than hospital acceptance.

8.9. Overall Test Results

"Demonstrated" means representative behavior was observed; "partial" means coverage or retained evidence is incomplete; "open" means validation has not been performed.

Table 8. 9. 1 Overall test results by test area and requirement reference.

No.	Test area	Requirement/acceptance reference	Current result	Evidence
1.	Backend rules	• FR-001 to FR-041 • FR-062 to FR-069 • AC-001 to AC-007 • AC-012	Core authentication, admin safety, node, asset, movement, and settings rules tested.	Core paths demonstrated; final case records required.
2.	Firmware	• FR-019, FR-024 • FR-052 to FR-056 • AC-004 • AC-009 • AC-010	Native/ESP32 helpers and modules tested; physical dependencies need bench checks.	Partial.
3.	Frontend and responsive UI	• FR-010 to FR-046 • FR-059 to FR-060 • FR-063 to FR-068 • NFR-019 to NFR-024 • AC-013	Main workflows and responsive states reviewed in both apps.	Representative evidence; automation limited.
4.	MQTT/API/WebSocket	• FR-019 • FR-024 • FR-040 • FR-047 to FR-061 • AC-004 • AC-007 to AC-011	Core command, detection, warning, and reconnect paths demonstrated.	Broader outage, ordering, load, and isolation tests pending.
5.	Failure/deployment	• ERR-001 to ERR-015 • DEP-001 to DEP-015 • AC-014 to AC-015	Error handling and environment separation exist.	Partial/open.
6.	Hospital UAT	• Relevant UR/UC groups • AC-013	Demonstrated in a simulation procedure, but not completed with representative users and signed criteria.	Open.

7.	Hosting and backup	• AC-012 • AC-14	Maintainers can download data as a backup for data migration at any time via PostgreSQL.	Monthly server data migration.

8.10. Remaining Defects

As a research prototype, the product is not a complete, readily deployed system.

The prototype still contains unresolved validation gaps and operational risks. The following table distinguishes residual issues from completed behavior and assigns priority according to whether an issue blocks a hospital pilot, a production deployment, or later maintainability.

Table 8. 10. 1 Remaining defects, unresolved uncertainties, and priorities.

No.	Area	Residual issue or uncertainty	Priority
1.	RFID installation	Real-site coverage and product casing are not verified yet.	High before pilot.
2.	Data longevity	The free PostgreSQL database can only be used for 30 days at most, so monthly data migration is very important. There might be another data server that is more reliable, but is still free to use.	High before pilot.
3.	Frontend consistency	Although both apps are maintained almost identically, status/count rules still need regression testing after changes.	Medium.
4.	Automated UI testing	Limited browser/component automation increases regression risk.	Medium.
5.	Security	Broker TLS, secret rotation, dependency scanning, and independent review remain open.	High before production.
6.	User validation	Formal hospital UAT and accessibility review remain open.	High before adoption.

9. Project Results and Evaluation

The project produced an integrated prototype rather than a standalone RFID reader or isolated web dashboard. Evaluation is therefore based on the extent to which the implemented subsystems cooperate to deliver the intended operational workflow and on the limitations of the evidence currently available.

9.1. Implemented Product

The implemented product includes configurable ESP32 Registration Desk and Checkpoint Nodes, MQTT event and command communication, a FastAPI backend, PostgreSQL persistence, a Registration App, and a Monitor App. It supports node lifecycle management, RFID-based asset registration, checkpoint location updates, controlled movement, alerts, activity history, authentication, role access, and live updates.

The product is deployable for demonstration and development through cloud-hosted web services while retaining local development configuration. Source code has been restructured toward smaller responsibilities, and detailed system behavior is documented through requirements, design diagrams, and testing materials.

9.2. Achievement of Project Objectives

Achievement was evaluated against the original project purpose and the requirements baseline rather than only against the existence of source code. The table records the extent to which each objective was achieved and identifies where additional evidence or real-site validation is still required.

Table 9. 2. 1 Achievement of the project objectives.

No.	Objective	Evaluation
1.	Register and deregister tagged assets.	Achieved in the prototype through a Registration Desk scan and protected backend workflow.
2.	Track last known checkpoint	Achieved for accepted checkpoint detections from configured nodes.
3.	Monitor node availability	Achieved through heartbeat, timeout, node status, and alerts.

4.	Control asset movement	Achieved through request, cancellation, approval/rejection, and destination-based completion.
5.	Warn about invalid movement	Achieved through flow-state resolution and unauthorized movement alerts.
6.	Provide live browser updates	Achieved through backend WebSocket events with API recovery.
7.	Protect actions by role.	Achieved at backend and frontend levels for implemented roles and safety policies.
8.	Demonstrate hospital-ready reliability	Partially achieved; site, long-duration, security, and formal UAT evidence remain incomplete.

9.3. Requirements Satisfaction

Assessment is made against the authoritative requirement groups rather than against an informal feature list. The prototype implements most mandatory functional behavior, but formal acceptance remains dependent on retained evidence for the related AC criteria. Non-functional "should" targets are evaluated separately so that production-readiness gaps remain visible.

Table 9. 3. 1 Requirements satisfaction assessment and evidence boundaries.

No.	Requirement group	Authoritative IDs	Assessment	Reason /evidence boundary
1.	Authentication, accounts, and role access	UR-001 to UR-004 UR-018 UR-020 FR-001 to FR-012 FR-062 to FR-063 FR-069 AC-001 to AC-002	Implemented; acceptance evidence incomplete	Core flows and safety rules exist, but the final System Test Report must retain all normal/invalid-role cases.
2.	Node management and availability	UR-005 to UR-007 UR-015 to UR-016 FR-013 to FR-021 FR-048 to FR-049 FR-054 AC-003 AC-004 AC-009	Implemented; partially verified	Lifecycle, heartbeat, timeout, and blink exist; device-scale, outage, and complete invalid-transition evidence remain partial.
3.	Asset registration lifecycle	UR-008 to UR-010 FR-022 to FR-030 AC-005	Implemented	Scan, registration, uniqueness, pending placement, and deregistration are supported for demonstrated scenarios.
4.	Movement and location integrity	UR-011 to UR-013 FR-031 to FR-041 FR-050 to FR-051 NFR-031 to NFR-034 AC-006 to AC-008	Implemented; formal coverage incomplete	Request, decision, destination completion, and wrong-location states exist; every alternative transition still needs retained evidence.
5.	Monitoring, alerts, activity, and live updates	UR-014 to UR-017 FR-042 to FR-061 AC-009 to AC-011	Implemented; partially verified	Both applications display API and WebSocket state; recovery and interruption testing is not yet exhaustive.
6.	System settings	UR-019 FR-064 to FR-068 UC-041 AC-012	Implemented according to role baseline	Admin and authorized Test User settings behavior must be verified against valid/invalid values and denied roles.
7.	Usability and responsiveness	NFR-019 to NFR-024 AC-013	Prototype-level	Manual reviews support the design, but formal task-based hospital UAT and accessibility evidence remain open.
8.	Reliability, deployment, and production security	NFR-007 to NFR-018 NFR-025 to NFR-030 NFR-035 to NFR-036 AC-014 to AC-015	Partially satisfied	Architecture and controls exist, while soak, failure injection, deployment repeatability, migration/restore, secret audit, and independent security review remain open.

9.4. System Demonstration Results

A representative demonstration covers UC-011 to UC-020 and AC-003 to AC-005: a node appears as discovered, is assigned to a role and room, responds to a blink command, and updates online/offline status; a tag is scanned at the Registration Desk, registered as an asset, and detected at the assigned checkpoint. The Monitor App reflects the accepted asset and node state without a manual page refresh under normal connected operation.

The movement demonstration covers UC-021 to UC-025 and AC-006 to AC-008: a request is created in the Monitor App, approved or rejected in the Registration App, the asset may be detected at a wrong checkpoint to show unauthorized movement, and a valid detection at the approved destination completes the movement. This sequence demonstrates that the product is based on controlled state transitions and accepted physical events, not only form updates.

The movement-request sequence begins in the Monitor App. The backend API and movement service validate the asset, destination, user permission, and absence of a conflicting pending request before storing the new request. After persistence succeeds, a WebSocket event makes the pending decision visible in the Registration App.

The approval sequence starts when an authorized registration-side user decides a pending request. The decision service verifies that the request is still eligible, records the approver and decision, updates the related asset flow state, and broadcasts the result to the Monitor App. Approval authorizes physical movement but does not complete the destination change.

The checkpoint-detection sequence connects the physical event to the final asset state. A Checkpoint Node publishes a tag observation through MQTT; the backend handler validates the node and tag, records the detection, and asks the flow resolver to compare the detected room with the assigned room and any approved destination. The resulting asset, movement, alert, and activity changes are persisted before both applications are notified.

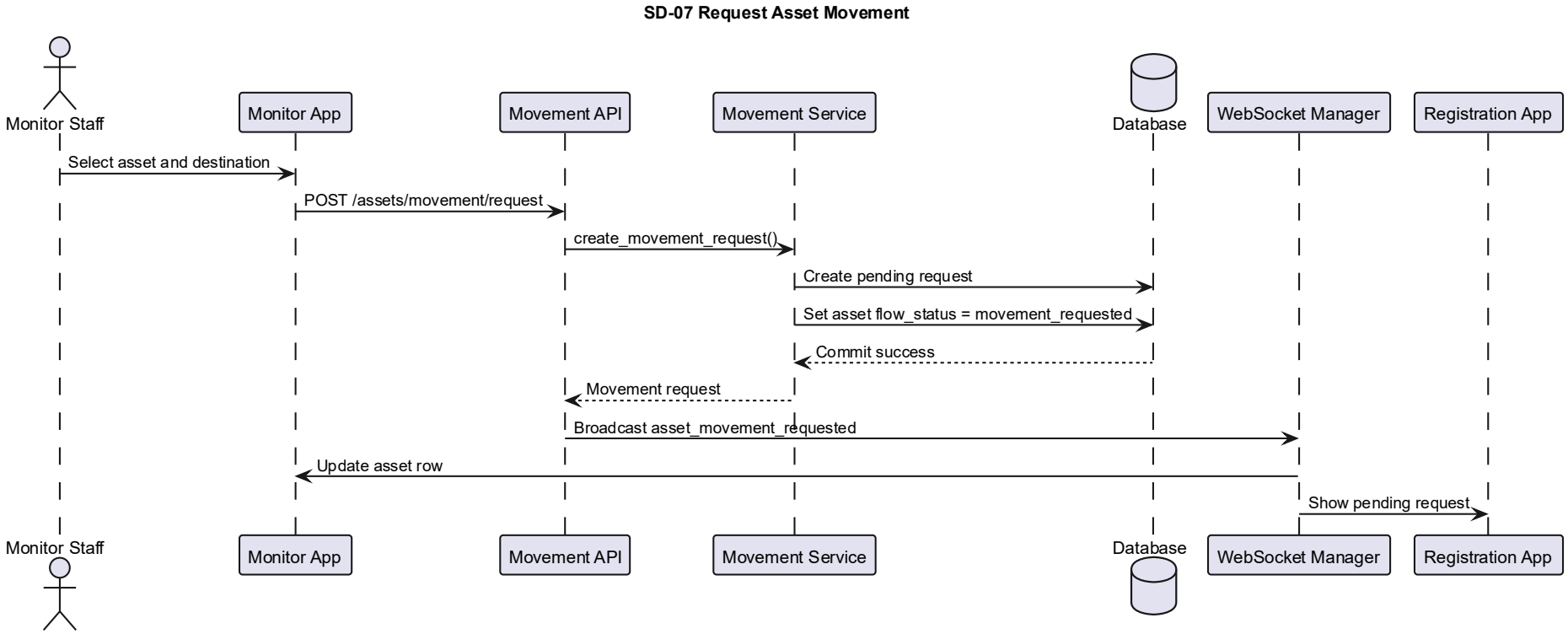

Figure 9. 4. 1 Sequence for creating an asset movement request and notifying the Registration App.

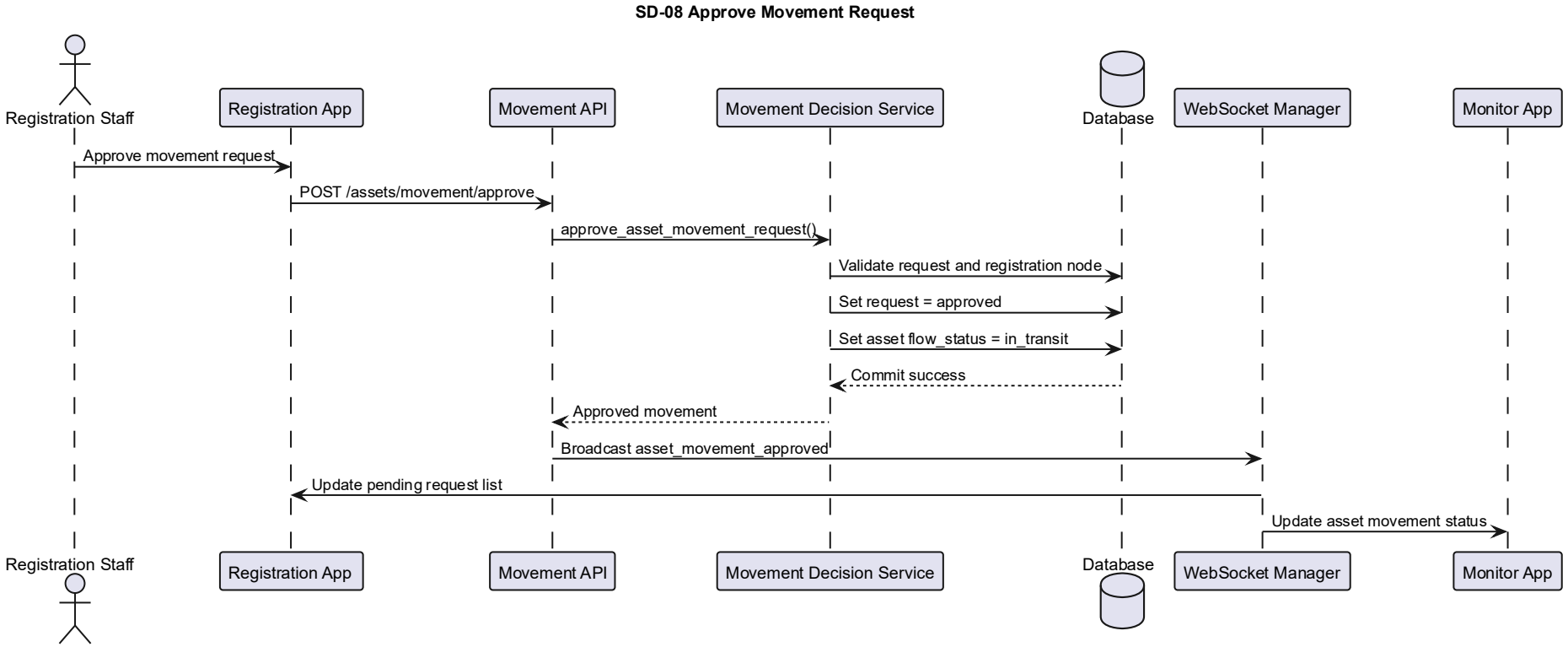

Figure 9. 4. 2 Sequence for deciding a movement request and notifying the Monitor App.

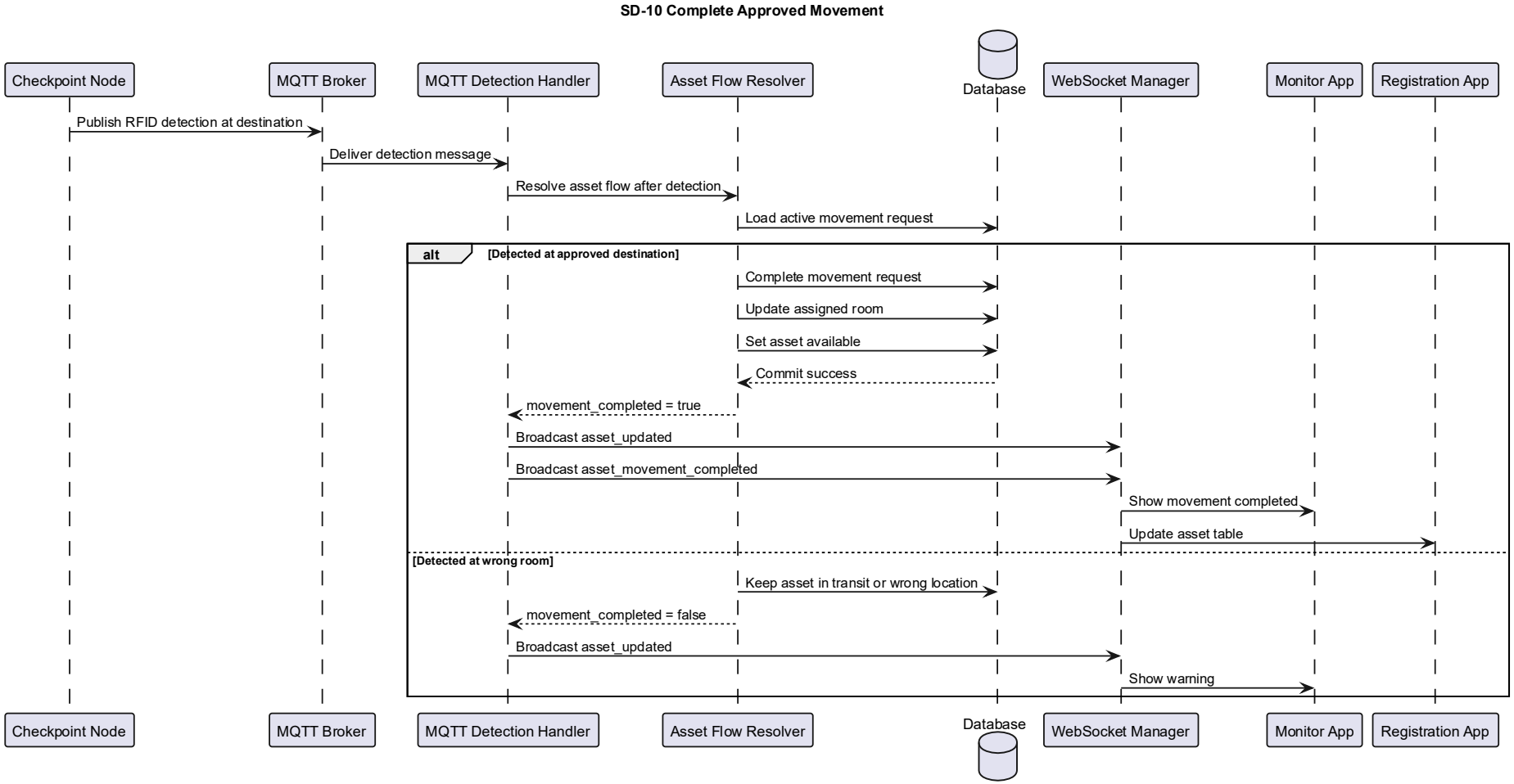

Figure 9. 4. 3 Sequence for processing a checkpoint detection, resolving asset flow, persisting results, and updating both applications.

9.5. Usability Evaluation

The two-app separation reduces cognitive load by keeping registration and administration actions away from the operational monitoring view. Status badges, role-aware navigation, loading indicators, confirmation behavior, search, sorting, and pagination make large tables more manageable. Responsive refinements support desktop, half-window, tablet, and mobile layouts, although data-heavy tables remain most effective on a larger screen.

Usability limitations include the number of possible states, the need to explain “last known” location, and the risk that users interpret a disconnected WebSocket indicator as a sensor-node failure. Clear wording, onboarding, legends, and a formal task-based UAT should be used to validate the interface with actual hospital roles.

9.6. Reliability Evaluation

The architecture includes multiple recovery mechanisms: Wi-Fi reconnect, MQTT reconnect and resubscribe, periodic heartbeat, backend offline detection, WebSocket reconnect, API reload after missed events, and transactional database operations. These mechanisms improve resilience and have solved several observed development defects.

Reliability is not yet proven for continuous hospital operation. The prototype needs soak testing, power interruption tests, broker and database outage scenarios, message duplication/out-of-order tests, many-node simulation, and physical RFID coverage validation. The current evaluation is therefore “functionally resilient in tested scenarios,” not “high availability.”

9.7. Security Evaluation

The strongest security property is that protected business actions are enforced by the backend, not only hidden in the UI. Password hashing, token validation, role dependencies, disabled-account handling, last-administrator protection, secret configuration, and safe error messages are included. Frontend session handling clears invalid tokens and prevents unauthorized navigation from being treated as successful access.

The remaining security gap is operational hardening. Production use requires broker encryption and authentication, secret rotation, dependency monitoring, audit retention, restricted API documentation, rate limiting where appropriate, backup protection, and an independent security assessment.

10. Discussion

The project demonstrates that the main challenge in hospital asset tracking is not the RFID read alone. Useful operational behavior emerges only when physical observations are connected to identity, expected location, movement authorization, node health, persistent history, and clear user responsibilities.

10.1. Technical Findings

While working on this project to answer the research question, the following technical findings have been identified.

- A checkpoint detection is meaningful only when the node has a stable identity and a valid assigned location.
- The backend must remain the single authority for state transitions; otherwise, firmware, database, and browser views can diverge.

- Movement approval and physical movement completion are different events and should not share one status.
- Live updates improve usability but cannot replace API-based state recovery after a disconnect.
- Heartbeat status is part of asset-tracking accuracy because an offline checkpoint creates uncertainty even when the rest of the platform is available.
- Responsive design for operational tables requires deliberate prioritization of columns and actions, not only smaller fonts.

10.2. Strengths of the Solution

The solution's primary strength is architectural separation. Sensor nodes observe, the backend validates and decides, PostgreSQL persists, and frontends present role-specific workflows. This separation makes security rules, movement state, and live updates more consistent than a design in which browsers or devices write operational state directly.

A second strength is the controlled movement model. The system distinguishes an expected room, a current detected room, a pending request, an approved movement, physical transit, wrong-location detection, and completed arrival. This provides more operational meaning than simply replacing one room value with another.

A third strength is maintainability work across all codebases. Firmware abstractions, backend services and repositories, frontend hooks and services, shared status utilities, and reduced file sizes improve the ability to test and change the system without altering unrelated behavior.

10.3. Limitations

Although this project mitigates the obstacles and challenges that most medical care providers face while managing asset movements within their facilities, some limitations still apply to this product. Those limitations are listed below:

- The system provides last-known checkpoint location, not continuous indoor positioning.
- RFID performance through Wi-Fi has been confirmed to be flexible enough to be used in a wide building area. But it has not been validated yet across a representative hospital layout, equipment materials, doors, and interference conditions.
- A checkpoint cannot report detections while its node, reader, network, broker, or backend path is unavailable.
- Cloud prototype hosting does not provide a hospital-grade service-level guarantee.
- Formal user-acceptance, accessibility, penetration, load, and long-duration reliability testing remain incomplete.
- The current product is not integrated with an authoritative hospital asset master, maintenance system, or procurement workflow.

10.4. Design Trade-Offs

The project prioritized a coherent, demonstrable workflow over precise localization. This made it possible to implement controlled movement and operational alerts within the available project scope, but it limits the certainty of an asset's position between checkpoints. The product also favors centralized backend rules over device autonomy, improving consistency at the cost of dependence on backend availability.

Separating the Registration and Monitor applications improves role focus, but creates duplicated concerns such as authentication, status display, shared components, and connection handling.

These concerns must be synchronized through shared conventions, tests, or eventually a common package. Cloud deployment improves accessibility for demonstration but introduces limitations from external services.

10.5. Differences Between Plan and Implementation

While making the product for this research, some changes to the plan have happened. Those changes are listed as follows:

Table 10. 5. 1 Differences between the original plan and the implemented system.

No.	Original or early assumption	Implemented direction	Reason for change
1.	Single main monitoring interface	Separate Registration App and Monitor App.	Registration/administration actions and operational monitoring have different users and risks.
2.	Simple tracker-to-gateway concept	Fixed Registration Desk and Checkpoint Node roles.	Known checkpoint identity is required to convert a detection into a meaningful room update.
3.	Direct location update	Movement request, decision, transit, unauthorized, and completion states.	Operational control requires authorization and physical confirmation.
4.	Hardcoded single-hospital settings	Hospital-scoped configuration and topic design.	Hardcoding would prevent safe multi-site expansion and could expose unrelated node data.
5.	Mostly monolithic feature files	Modular services, hooks, components, and utilities.	Large files became difficult to test, review, and maintain.

10.6. Lessons Learned

Developing the Hospital Fixed-Asset Tracking System showed that the most difficult part of an embedded tracking product is not reading an RFID tag in isolation. The greater challenge is maintaining one consistent interpretation of each event across the firmware, MQTT broker, backend services, database, WebSocket connection, and two browser applications. A detection may appear simple at the hardware level. However, the system must still determine whether the node is valid, whether the tag belongs to an active asset, whether the detected room is expected, whether an approved movement exists, and which asset, movement, alert, and activity records must be updated. This confirmed the importance of treating the backend as the authoritative processing boundary.

Another important lesson was that the system states must be defined before they are presented in the user interface. Early versions of the movement workflow did not clearly distinguish between a request being created, approved, physically in progress, or completed. The final design separates these stages and completes a movement only after the asset is detected at the approved destination. Similar clarification was required for node states such as discovered, assigned, online, offline, and disabled. Explicit state transitions reduced ambiguity in the backend and made the frontend status badges and available actions easier to implement consistently.

The project also demonstrated that real-time communication does not remove the need for persistent recovery mechanisms. WebSocket improves responsiveness, but a browser can temporarily disconnect and miss an event. MQTT allows lightweight communication with the sensor nodes, but the broker or Wi-Fi connection can also be interrupted. For this reason, the system combines WebSocket updates with API-based reloading, MQTT reconnection with resubscription, heartbeat monitoring with timeout detection, and database persistence as the source of truth. Live communication should improve the user experience without becoming the only mechanism for recovering the correct state.

Testing across subsystem boundaries proved more valuable than testing each component independently. Unit tests were useful for authentication, role restrictions, administrator safety, node

assignment, asset registration, movement rules, and firmware helpers. However, several important defects appeared only when the complete workflow was exercised. These included inconsistent online and offline states between applications, movement actions remaining unavailable after rejection, duplicate submissions, missed updates after WebSocket interruption, and differences between dummy-node behavior and normal heartbeat rules. This showed that embedded-and-web projects require both isolated tests and repeatable integration scenarios.

The frontend work showed that responsive design is not only a visual improvement. Tables containing node identities, asset locations, statuses, timestamps, and actions can become difficult to use when the browser is displayed in a half-window or on a mobile device. Responsive behavior therefore had to be considered together with workflow priority, column importance, sticky positioning, modal behavior, loading feedback, and role-based actions. A technically correct application can still fail operationally when users cannot quickly understand what is happening or what action is available.

Modularity also became increasingly important as the project grew. Large page components, backend services, MQTT handlers, and firmware modules were initially easier to create, but became harder to test and maintain as more rules were added. Dividing the implementation into focused routes, services, repositories, hooks, components, utilities, and hardware abstractions improved readability and reduced unintended changes. The lesson is that refactoring should not be postponed until the end of a multi-component project; it should occur when responsibilities begin to overlap, or a file becomes difficult to reason about.

Finally, the project confirmed that a functioning prototype is different from a production-ready hospital system. The current product demonstrates the intended architecture and central workflows. Still, production use requires physical RFID coverage validation, reliable electrical installation, formal hospital user acceptance, longer-duration reliability tests, stronger MQTT security, monitored backups, recovery drills, and a more dependable hosting environment. Clearly communicating these boundaries is necessary to avoid overstating the evidence produced by the project.

11. Safety, Privacy, and Ethical Considerations

Although the product does not make clinical decisions, it operates around hospital equipment and may influence staff expectations about availability. Safety and ethics therefore require clear boundaries, secure operation, and honest representation of location certainty.

11.1. Electrical and Installation Safety

ESP32 boards, RFID readers, antennas, power supplies, and wiring must be enclosed and installed so they do not create exposed conductors, unstable connectors, trip hazards, blocked doors, or interference with hospital equipment. Power supplies must provide the required voltage and current with adequate regulation. Bench prototypes and breadboard power arrangements are not suitable for unattended hospital installation.

Installation should follow facility electrical and infection-control requirements. Any enclosure, mounting, or cabling near a clinical area must be reviewed by hospital engineering or facilities staff before use.

11.2. RFID Installation Considerations

UHF RFID readers and antennas must be positioned to achieve the intended checkpoint zone without creating excessive overlap with adjacent rooms. Tag mounting should be tested on the

actual asset material and location. Reader power should be configured conservatively and increased only through controlled tests. A deployment should document blind spots, duplicate-read handling, and the operational meaning of a missed checkpoint.

11.3. User Account Privacy

The system stores account identity, role, status, and activity needed to authorize and audit actions. It should not collect unrelated personal or clinical information. Passwords are stored only as hashes. Tokens and credentials must not be logged, exposed in screenshots, embedded in source code, or shared through public demonstration accounts with write access.

11.4. Access Control

Least-privilege access should be applied. Monitoring staff needs operational visibility and movement requests, registration staff needs registration and decision functions, technicians need node management, and administrators need user and global configuration control. The backend must enforce this matrix regardless of what the frontend displays. Demonstration accounts should be read-only or tightly restricted and separate from administrator credentials.

11.5. Data Minimization

The prototype should store only the information required to identify assets, nodes, locations, movement decisions, alerts, and important activity. Patient identity, staff location, clinical measurements, and treatment information are outside scope. Logs should avoid raw secrets and should use retention rules so technical history does not grow indefinitely without purpose.

11.6. Clinical-System Boundary

The application is an operational support tool, not a medical device controller or clinical decision system. A displayed location is the last accepted checkpoint observation and may be outdated when coverage or connectivity is unavailable. Users must not treat the interface as proof that equipment is safe, calibrated, sterile, available for use, or physically present without local verification.

12. Maintenance and Future Development

The prototype can be continued only if source code, configuration, deployment, database changes, test evidence, and diagrams are maintained together. The following approach separates routine maintenance from product expansion.

12.1. Maintenance Approach

Each codebase already has a concise README containing the purpose, prerequisites, environment variables, and build/run commands. However, testing commands, deployment notes, and project structure can be added and improved further during further maintenance. Changes should use version control, reviewed commits, and a release note that identifies database migrations, firmware compatibility, API changes, and required frontend configuration. Large modules should continue to be split by responsibility rather than by arbitrary line count alone.

Everything needed for further maintenance, however, is already listed in the system design document, use case document, and system requirement document.

12.2. Monitoring and Logging

Operational monitoring should include backend health, database connectivity, MQTT connection, WebSocket errors, node heartbeat age, command acknowledgment failures, repeated invalid messages, and unresolved alerts. Logs should also be downloadable separately by monitoring users, not only by maintainers or technicians, only during data migration.

12.3. Backup and Recovery

For PostgreSQL, data should be exported on a defined schedule and before service expiry or migration. Recovery instructions must include creation of the target database, environment-variable update, schema migration, data import, backend restart, and verification of users, nodes, assets, movement requests, and timestamps. Restore procedures should be tested; an untested backup should not be treated as a recovery solution.

However, for further maintenance and advanced deployments, it is recommended to use a bigger and more reliable free environment database. The PostgreSQL being used right now is apparently the limited 30-day free database, which requires data maintenance and migration every 29 to 30 days. This is only good for lab research and prototypes, not for a fully product-ready deployment.

12.4. Scalability

The architecture can support more assets and nodes because sensors publish independent events and the backend uses persistent identifiers. Scaling still requires database indexes, connection limits, MQTT broker capacity, WebSocket fan-out, background-task coordination, and event-rate testing. Hospital-scoped topic namespaces and data filters are necessary so one hospital does not receive another hospital's nodes or assets.

At larger scale, asynchronous job handling, message deduplication, idempotent event processing, centralized metrics, and a stronger deployment platform may be needed. These changes should be based on measured load rather than introduced prematurely.

12.5. Recommended Improvements

The recommended improvements are prioritized according to their effect on deployment safety, verification quality, operational reliability, and maintainability. High-priority actions should be completed before a hospital pilot or production use, while lower-priority enhancements can follow after the core system is validated in a representative environment.

Table 12. 5. 1 Prioritized recommendations for maintenance and future development.

No.	Priority	Improvement	Expected benefit
1.	I	Run a real-site RFID survey and pilot with representative assets and locations.	Define detection zones, blind spots, and installation rules.
2.	II	Complete soak, outage, power-cycle, broker, database, and WebSocket recovery tests.	Provide operational reliability evidence.
3.	III	Use per-hospital/per-node MQTT credentials, TLS, and topic authorization.	Reduce cross-site access and message-injection risk.
4.	IV	Add automated frontend component and end-to-end browser tests.	Reduce role, responsive-layout, and movement regressions.
5.	V	Formalize a read-only demo role and sanitized data.	Support safe portfolio and stakeholder review.
6.	VI	Automate monitored backup, migration, and deployment.	Reduce service-expiry and manual-deployment risk.
7.	VII	Conduct formal hospital UAT and accessibility review.	Validate tasks, terminology, and usability with real users.

12.6. Potential Hospital-System Integration

Future integration should begin with an authoritative asset identifier and location master rather than duplicating all hospital data. The tracking backend could expose or consume APIs for asset

identity, department, room, maintenance status, and decommissioning. Movement events could be published to an integration layer instead of writing directly into an HIS or ERP database.

Integration requires data ownership, error reconciliation, audit rules, authentication between systems, and a clear clinical boundary. The prototype should remain usable as an independent service until these contracts and responsibilities are formally defined.

13. Conclusion

This chapter concludes the Hospital Fixed-Asset Tracking System project by summarizing the implemented product, assessing the achievement of its objectives, evaluating the prototype as a complete system, and identifying the most important next steps. The conclusion is based on the implementation and evidence currently available and therefore distinguishes demonstrated prototype functionality from production readiness.

13.1. Project Summary

This Hospital Fixed-Asset Tracking System project addressed the difficulty of maintaining dependable visibility of movable hospital assets when equipment is transferred between rooms without an automatically updated record. To address this problem, a checkpoint-based tracking prototype was designed and implemented using passive UHF RFID tags, an ESP32-based Registration Desk and Checkpoint Nodes, MQTT communication, a FastAPI backend, PostgreSQL persistence, WebSocket live updates, and two React browser applications.

The Registration App supports first-administrator setup, authentication, role-aware navigation, node management, asset registration and deregistration, movement-request decisions, user management, account security, and supported system settings. The Monitor App provides operational visibility through dashboard summaries, asset and node monitoring, alerts, activity records, and movement-request creation or cancellation. The two applications have different responsibilities but use the same backend-controlled data and business rules.

The embedded nodes act as event sources. The Registration Desk Node reports RFID tags used during registration or deregistration of the hospital asset, while Checkpoint Nodes report asset detections and periodic heartbeat messages. The firmware does not independently determine whether an asset is registered, correctly located, or authorized to move. Instead, the backend validates the node and tag, applies the asset and movement rules, persists the resulting state, creates alerts or activity records where required, and broadcasts accepted changes to connected browser clients.

The completed prototype supports a controlled asset-movement process. A monitoring user creates a movement request, a registration-side user approves or rejects it, and approval places the asset into an authorized movement state. The movement is completed only after a valid checkpoint detects the asset at the approved destination. A detection at another location does not complete the request and may produce an unauthorized-movement warning. This workflow connects user authorization with a physical RFID event rather than treating a form submission as proof that the asset has arrived.

The project also produced supporting requirements, use-case, design, testing, and report documentation. Together, these materials describe what the system is expected to provide, how its actors interact with it, how the components are structured, how important states and communication paths behave, and which verification activities have been completed or remain open.

13.2. Achievement of Objectives

The principal objective of creating an integrated hospital asset-tracking prototype was achieved. The project produced working firmware, backend services, database persistence, a Registration App, and a Monitor App that cooperate through defined MQTT, HTTPS, WebSocket, serial, and database interfaces.

The asset-management objective was achieved through RFID-based registration and deregistration, active-tag uniqueness checks, assigned-room selection, checkpoint detection, last-known-location updates, and asset flow states. Newly registered assets remain pending placement or on the move until detected in their assigned room, preventing registration alone from being interpreted as confirmed physical placement.

The node-management objective was achieved through discovery, assignment, editing, role and location configuration, status monitoring, blink identification, heartbeat processing, offline detection, and controlled unassignment or deletion. The shared firmware structure allows Registration Desk and Checkpoint roles to use the same main modules with configuration-dependent behavior.

The controlled-movement objective was achieved through request, cancellation, approval, rejection, transit, unauthorized-movement, and destination-completion behavior. The backend prevents conflicting pending requests and preserves the distinction between authorization to move and confirmation that the movement has physically completed.

The monitoring objective was achieved through dashboards, searchable and sortable asset and node views, status badges, warning summaries, alerts, activity records, and WebSocket-driven updates. The application can communicate important conditions such as an unknown RFID tag, an offline node, or an asset detected outside its expected location.

The security and access-control objective was substantially achieved at prototype level. The backend verifies accounts, passwords, active status, tokens, and roles; passwords are hashed; protected routes enforce permissions; disabled users are rejected; and administrator safety rules protect the final active administrator. Frontend route protection and hidden actions improve usability but remain secondary to backend enforcement.

The maintainability objective was addressed by separating firmware responsibilities, backend routes and services, React pages and hooks, API modules, utilities, and reusable UI components. Automated backend and firmware tests provide evidence for several critical rules. However, frontend automation, complete formal integration evidence, and retained test records for every acceptance criterion remain incomplete.

The objective of demonstrating production-level reliability has not yet been fully achieved. The project has implemented recovery mechanisms and demonstrated representative scenarios. Still, broader soak testing, power interruption, broker and database outages, multi-node load, real-site RFID coverage, formal hospital UAT, and independent security review are still required. The objectives should therefore be considered achieved for an integrated prototype, but only partially achieved for operational hospital deployment.

13.3. Final Evaluation

The final result is a technically coherent and functional prototype that demonstrates how checkpoint-based RFID tracking can support hospital asset visibility and controlled movement. Its strongest design decision is the separation of sensing, processing, persistence, and presentation. ESP32 nodes report observations, the MQTT broker transports messages, the FastAPI backend applies authoritative rules, PostgreSQL retains state, and the two browser applications present role-specific workflows. This separation makes the product easier to understand, test, extend, and secure than a design in which the nodes or frontends directly modify asset state.

The system also provides more operational meaning than a simple “last scanned room” application. By maintaining assigned room, current room, movement-request state, flow status, latest detection, node availability, alerts, and activity, the prototype can distinguish an asset that is correctly placed from one that is awaiting placement, approved for movement, or present in an unauthorized location. This state-based design is one of the project’s most important results.

The prototype is suitable for technical demonstration, continued development, controlled laboratory testing, and preparation for a hospital pilot. It is not yet suitable for unrestricted production deployment. Location accuracy depends on RFID hardware, tag placement, antenna configuration, environmental interference, network coverage, and installation design that have not been fully validated in a real hospital. The current hosting and database arrangement is also appropriate for prototype access but does not provide the availability, backup assurance, or service continuity expected from a hospital operational system.

The evaluation must therefore remain balanced. The project successfully proves that the selected hardware and software architecture can support the intended registration, tracking, movement, warning, and monitoring workflows. It also identifies the remaining engineering work honestly. The result should be judged as a strong integrated prototype with a credible path toward a pilot, rather than as a finished commercial or clinically certified product.

13.4. Recommendations

The first recommendation is to conduct a controlled RFID site survey and hospital pilot. Representative assets should be tested with their actual tag mounting, material, orientation, movement speed, doorway geometry, nearby metal, people, and potential overlapping read zones. The pilot should establish reliable checkpoint boundaries and identify locations where antenna placement, reader power, shielding, or multiple antennas are required.

The second recommendation is to complete a formal System Test Report against the defined requirements, use cases, and AC-001 to AC-015 acceptance criteria. Each test should record the environment, version, input data, expected result, actual result, evidence, defect reference, corrective action, and retest result. Particular attention should be given to invalid transitions, duplicate MQTT messages, out-of-order detections, power cycles, network interruption, broker failure, database unavailability, WebSocket reconnection, and longer-duration operation.

Before production deployment, the MQTT and deployment security model should be strengthened. Nodes should use authenticated and encrypted broker connections where supported, credentials should be scoped per hospital or per node, and topic authorization should prevent one deployment from publishing or subscribing to another hospital’s data. Secrets should support rotation, production API documentation should be restricted, dependencies should be monitored, and an independent security review should be performed.

The database and hosting environment should also be replaced or upgraded before operational use. Automated backups, tested restoration, migration procedures, service monitoring, health

checks, and alerting should be introduced. A production database should not depend on repeated manual migration caused by a short-lived free service. The selected platform should provide appropriate availability, resource limits, retention, and access control for the intended deployment.

Automated frontend testing should be expanded to cover authentication, role-specific navigation, node and asset workflows, movement requests and decisions, status normalization, WebSocket reconnect behavior, and responsive layouts. These tests would reduce the risk that changes to one application produce inconsistent behavior in the other.

Formal user-acceptance and accessibility testing should be conducted with representative administrators, technicians, registration staff, and monitoring staff. The evaluation should measure whether users can complete important tasks, understand the difference between assigned and current location, interpret movement and connection states, respond to alerts, and recover from errors without needing knowledge of MQTT, WebSocket, or database behavior.

After the independent tracking service has been validated, integration with a hospital information, facilities, or asset-management system may be considered. Such integration should use controlled APIs or an integration layer with clearly defined data ownership, identifiers, authentication, audit requirements, and error reconciliation. Direct access to another system's database should be avoided.

The recommended development order is therefore: validate the RFID installation, complete formal system and recovery testing, harden security and deployment, perform hospital UAT, and only then expand into larger-scale deployment or hospital-system integration. This sequence preserves the project's current strengths while addressing the areas that present the greatest operational risk.

14. Appendices

These appendices provide compact references for the report. They do not replace the detailed System Design Document, current backend route definitions, source-code enums, or System Test Report. Values should be verified against the release being submitted.

14.1. Diagram List

Only the following existing files from all_design(2).zip are needed in the report. Detailed diagrams already contained in the System Design Document should not be duplicated unless they appear in this list. No new diagram needs to be created.

14.2. Status Value Reference

The following values summarize the operational states described in this report. The exact spelling and storage rules must match the submitted source code and System Design Document.

Table 14. 2. 1 Reference list of node, asset, movement, and alert status values.

No.	Category	Value	Meaning
1.	Node	DISCOVERED	Backend knows the stable device identity, but an authorized assignment is not complete.
2.	Node	ASSIGNED	The node has an accepted role and required hospital/location configuration but is not currently confirmed online.
3.	Node	ONLINE	A valid recent heartbeat confirms that the node is available.
4.	Node	OFFLINE	The heartbeat age exceeded the configured timeout or connectivity was otherwise lost.
5.	Node	DISABLED	The node record remains present but operational processing/actions are restricted.
6.	Node	UNKNOWN	Frontend fallback for unexpected or missing status; not a preferred stored lifecycle value.
7.	Asset lifecycle	ACTIVE	The asset and tag are currently registered for tracking.
8.	Asset lifecycle	DEREGISTERED / INACTIVE	The asset is no longer active in the tracking workflow.
9.	Asset flow	IN_PLACE	The asset is detected at its assigned room.
10.	Asset flow	PENDING_APPROVAL	A movement request exists but has not been decided.
11.	Asset flow	ON_THE_MOVE	The asset is newly registered, awaiting placement, or has an approved movement awaiting destination detection.
12.	Asset flow	UNAUTHORIZED	A detection conflicts with the assigned room and no valid destination completion applies.
13.	Movement	PENDING	The request is waiting for a registration-side decision.
14.	Movement	APPROVED	Movement is authorized but not yet physically completed.
15.	Movement	REJECTED	The request was closed without authorizing movement.
16.	Movement	CANCELLED	The requester canceled an eligible pending request.
17.	Movement	COMPLETED	The asset was detected at the approved destination and assignment was updated.
18.	Alert	OPEN	The warning requires attention or has not yet been resolved.
19.	Alert	RESOLVED	The underlying condition has been cleared according to the implemented policy.

14.3. MQTT Topic Reference

Topics are hospital-scoped so that devices and backend subscriptions can be isolated by deployment. The exact root, slug rules, broker settings, and authorization policy must be verified against the current configuration. The frontend applications do not connect directly to MQTT.

Table 14. 3. 1 Representative MQTT topic patterns and message contents.

No.	Message category	Representative topic pattern	Typical content
1.	Registration scan	hospital/{hospital_id}/nodes/{node_id}/registration/scan	Message type, node ID, tag ID, timestamp.
2.	Checkpoint detection	hospital/{hospital_id}/nodes/{node_id}/rfid/detection	Message type, node ID, tag ID, detected time, reader information where included.
3.	Heartbeat/status	hospital/{hospital_id}/nodes/{node_id}/heartbeat	Node ID, role/status, firmware/runtime information, timestamp.

4.	Backend command	hospital/{hospital_id}/nodes/{node_id}/commands	Command ID, command name such as blink, target information, optional duration.
5.	Command acknowledgement	hospital/{hospital_id}/nodes/{node_id}/commands/ack	Command ID, node ID, success/failure result, message, timestamp.

14.4. WebSocket Event Reference

WebSocket events notify authenticated browser clients after the backend has accepted and persisted an operational change. Event names and payload fields must remain consistent with the backend broadcaster and the socket handlers in both applications. In contrast, HTTP APIs remain the recovery source after reconnection or a suspected missed event.

Table 14. 4. 1 Representative WebSocket event types, producers, and effects.

No.	Event type	Typical producer	Consumers/effect
1.	registration_scan	Accepted Registration Desk MQTT scan handler.	Registration App receives the scanned tag for register/deregister workflow.
2.	asset_updated	Asset registration, deregistration, location, or movement service.	Registration and monitor asset state refresh.
3.	node_updated	Assignment, heartbeat, offline monitor, disable/enable service.	Node tables and dashboard counts update.
4.	movement_request_updated	Movement create, cancel, approve, reject, or completion service.	Registration request panel and Monitor asset actions update.
5.	alert_created / alert_updated	Unknown tag, unauthorized movement, offline node, or resolution service.	Dashboard warning count and Alerts page update.
6.	activity_created	Important business or operational event.	Activity page appends or refreshes the event.
7.	command_acknowledged	MQTT acknowledgment handler.	Node blink or command feedback is shown to the initiating user.
8.	connection/status event	Backend or socket lifecycle.	Connection indicator and recovery behavior update.

14.5. API Endpoint Summary

The table summarizes endpoint families rather than serving as the authoritative API contract. Exact paths, methods, schemas, and role dependencies must be verified against the submitted FastAPI route modules or generated OpenAPI specification.

Table 14. 5. 1 API endpoint families, representative operations, and primary callers.

No.	API area	Representative operations	Primary roles/callers
1.	Authentication	First-admin setup, login, current session/user, password change, logout/session clearing behavior.	Unauthenticated setup/login; authenticated users for account actions.
2.	Users	List, create, update role/profile, activate/deactivate, delete where allowed.	Admin; last-active-admin safety policy enforced.
3.	Nodes	List/discover, assign, edit, unassign, disable/enable, delete eligible record, blink command, status.	Admin and technician; monitoring roles may have read-only access.
4.	Assets	List, register from scanned tag, deregister, detail/status, current location.	Registration roles for write; permitted roles for read.
5.	Movement requests	List, create, cancel, approve, reject; completion occurs through accepted detection.	Monitor roles request/cancel; registration roles approve/reject.
6.	Alerts	List/filter, resolve or acknowledge where implemented.	Monitoring and authorized administrative roles.
7.	Activity	List/filter operational event history.	Monitoring and authorized administrative roles.
8.	Settings	Read/update hospital, MQTT, or system configuration values where enabled.	Admin and authorized Test User according to FR-064 to FR-068 and UC-041.
9.	WebSocket	Authenticated live event connection for each frontend.	Valid sessions with application access.
10.	Health / operational status	Backend, database, MQTT, or service health where exposed.	Deployment monitoring; public exposure should be minimized.

References

1) *EL-UHF-RMT01. (n.d.). https://electron.id/produk/el-uhf-rmt01/*
2) *EL-UHF-RMT01 menggunakan mikrokontroler. (n.d.). https://electron.id/tutorial/el-uhf-rmt01-mikrokontroler/*
3) *Afiyat, N., Navilla, R. H., & Hariyadi, M. (2023). IoT-based infusion fluid monitoring system using the MQTT protocol. Jurnal Nasional Teknik Elektro dan Teknologi Informasi, 12(1). https://doi.org/10.22146/jnteti.v12i1.5862*
4) *Bednarz, B., & Miłosz, M. (2025). Benchmarking the performance of Python web frameworks. Journal of Computer Sciences Institute, 36, 336–341. https://doi.org/10.35784/jcsi.7738*
5) *Santiago, S., & Benisius, B. (2025). Testing performa framework Express.js, Gin, dan FastAPI menggunakan API dan JMeter. Journal of Information System, Applied, Management, Accounting and Research, 9(3), 1219–1226. https://doi.org/10.52362/jisamar.v9i3.1986*
6) *React. (n.d.-a). React: The library for web and native user interfaces. Retrieved August 3, 2026, from https://react.dev/*
7) *React. (n.d.-b). Built-in React Hooks. Retrieved August 3, 2026, from https://react.dev/reference/react/hooks*
8) *React. (n.d.-d). Sharing state between components. Retrieved August 3, 2026, from https://react.dev/learn/sharing-state-between-components*
9) *Tong, J., Jikson, R. R., & Gunawan, A. A. S. (2023). Comparative performance analysis of JavaScript frontend web frameworks. In 2023 3rd International Conference on Electronic and Electrical Engineering and Intelligent System (ICE3IS). IEEE. https://doi.org/10.1109/ICE3IS59323.2023.10335250*
10) *Tinggi, K. R. T. D. P. (n.d.). Garuda - Garba rujukan digital. https://garuda.kemdiktisaintek.go.id/documents/detail/4530770*
11) *Selamet, I. (2023, July 27). Ratusan Aset milik RSUD Cianjur Hilang, Kemana larinya? Detikjabar. https://www.detik.com/jabar/berita/d-6845016/ratusan-aset-milik-rsud-cianjur-hilang-kemana-larinya*
12) *Agustian, B. (2025, July 8). Gubernur Babel laporkan kasus hilangnya 46 alat kesehatan senilai Rp15 miliar ke polda. Antara News. https://babel.antaranews.com/berita/498305/gubernur-babel-laporkan-kasus-hilangnya-46-alat-kesehatan-senilai-rp15-miliar-ke-polda*
13) *Pelaku Pencurian Alkes di UPTD Puskesmas Oinlasi Dibekuk Polisi. (n.d.). katakini.com. https://www.katantt.com/artikel/46011/pelaku-pencurian-alkes-di-uptd-puskesmas-oinlasi-dibekuk-polisi/*
14) *Redaksi. (2026, June 29). Direktur RSUD Pandan lapor polisi kasus pencurian sparepart alkes. Harian SIB. https://www.hariansib.com/Marsipature-Hutanabe/446254/direktur-rsud-pandan-lapor-polisi-kasus-pencurian-sparepart-alkes/all/*